

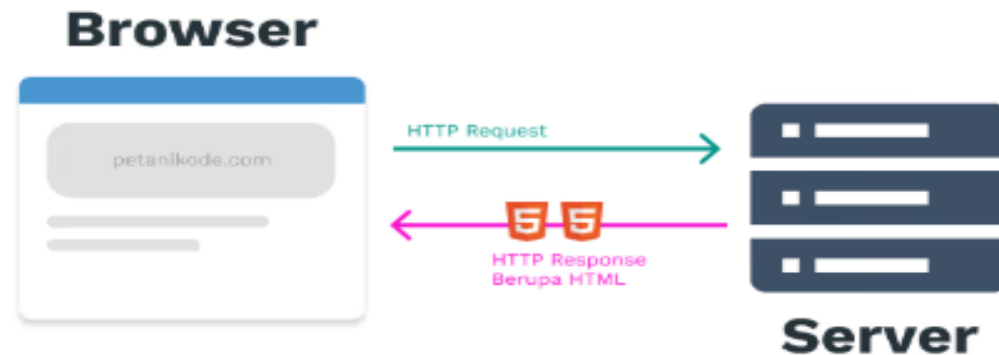
Dasar Pemrograman Web

Materi 4 Pemrograman Web

Wahyudi, S.Kom

Konsep Dasar Pemrograman Web

- Asal-usul Kemunculan PHP :
- Pada zaman dulu.. Web itu statis, cuma dibuat dengan HTML dan sedikit sentuhan CSS.
- Waktu itu memang teknologi web masih belum secanggih sekarang.
- Jadi saat orang membuka web, mereka sebenarnya cuma membuka file HTML dari web server.



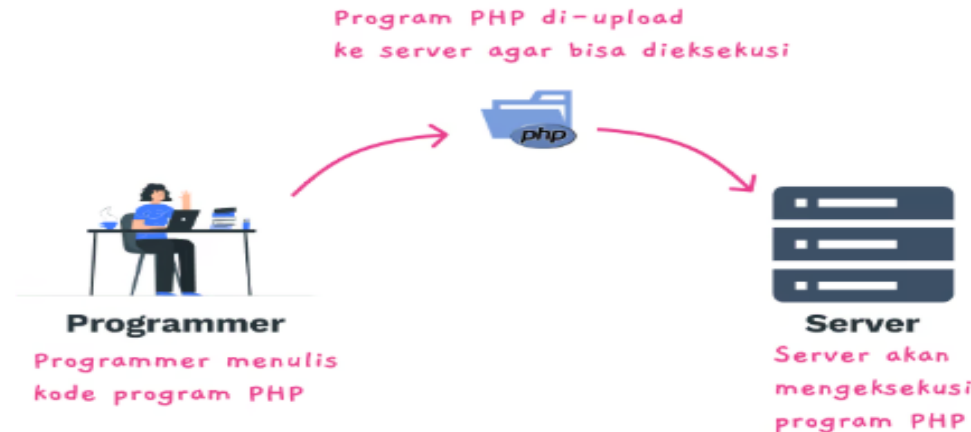
- Masalah waktu itu:
- Jika webnya statis, kita nggak bisa tau siapa saja yang buka dan berapa orang yang buka.
- Karena itu, di tahun 1993.. Rasmus Lerdorf membuat bahasa pemrograman PHP.
- Awalnya PHP cuma dipakai sendiri oleh Rasmus Lerdorf, kemudian di-rilis ke publik di tahun 1995.
- Dulu PHP dikenal sebagai singkatan:
- PHP = *Personal Home Page*.
- Tapi makin ke sini, singkatan itu berubah menjadi:
- PHP = *PHP: Hypertext Preprocessor*.

Apa itu PHP?

- PHP adalah bahasa pemrograman *scripting* yang berjalan pada server. PHP dikembangkan dengan bahasa C sehingga sintaksnya mirip-mirip dengan bahasa C.
- **Peralatan yang dibutuhkan untuk Coding PHP**
- Ada 4 alat yang perlu disiapkan untuk mulai coding PHP:
 - 1.**PHP interpreter** – program buat menjalankan PHP pada server
 - 2.**Teks Editor** – buat menulis kode PHP
 - 3.**Web Server** – service buat menjalankan PHP
 - 4.**Web Browser** – buat melihat atau membuka website

Alur Membuat Program PHP

- Alur membuat program PHP dimulai dengan menulis program PHP. Setelah itu, menjalankannya di server.
- Coba perhatikan gambar ini:

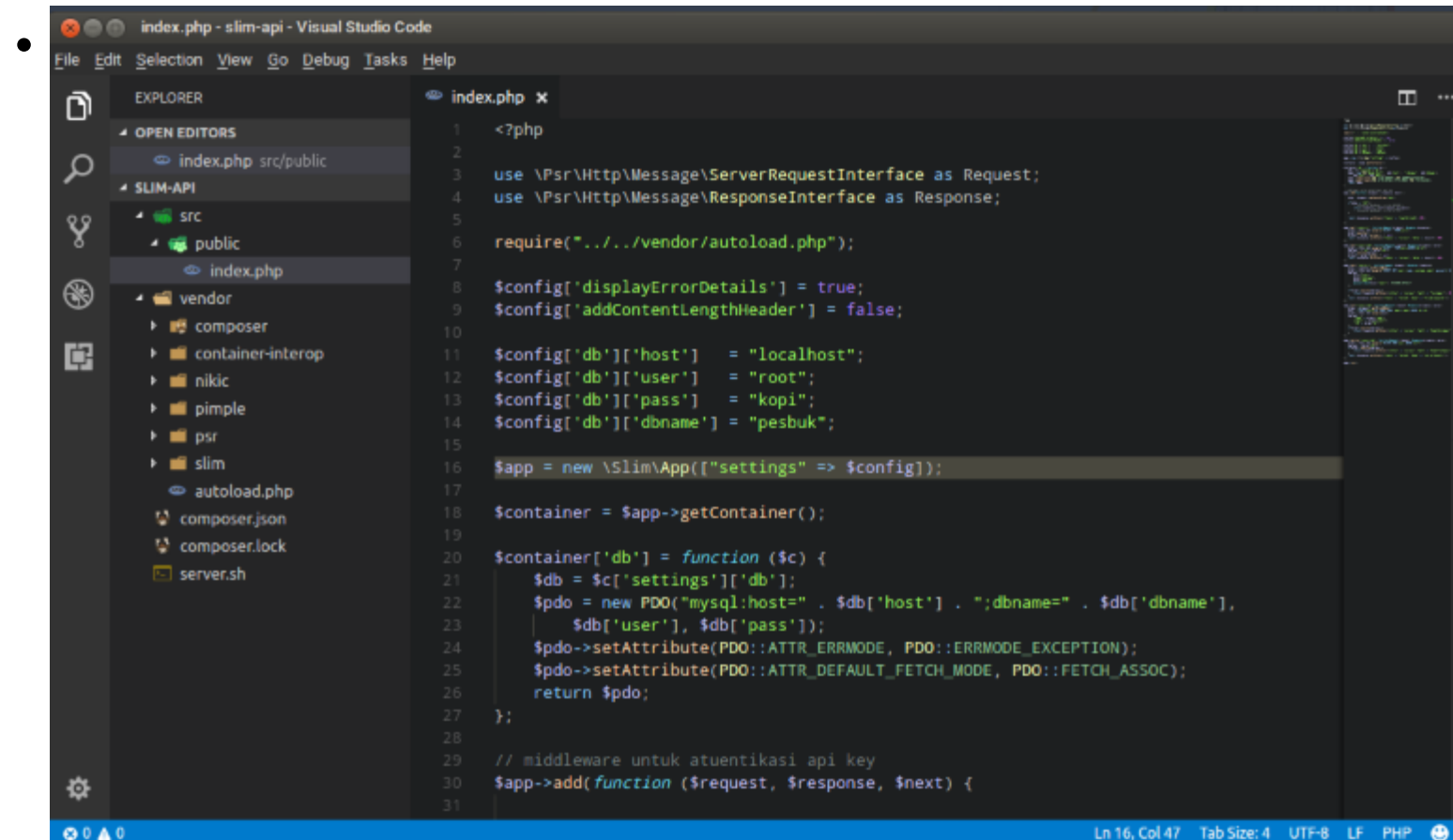


- Kita sebagai programmer menulis program PHP, kemudian kita upload ke server setelah program itu jadi.
- Selanjutnya, saat pengunjung atau user membuka web.. Dia akan menerima HTML yang merupakan hasil dari eksekusi PHP.



1. Teks Editor

- Teks Editor akan kita gunakan untuk menulis kode PHP.
- Ada banyak pilihan teks editor di Linux seperti: Gedit, Geany, Atom, Sublime, CodeLite, dsb.



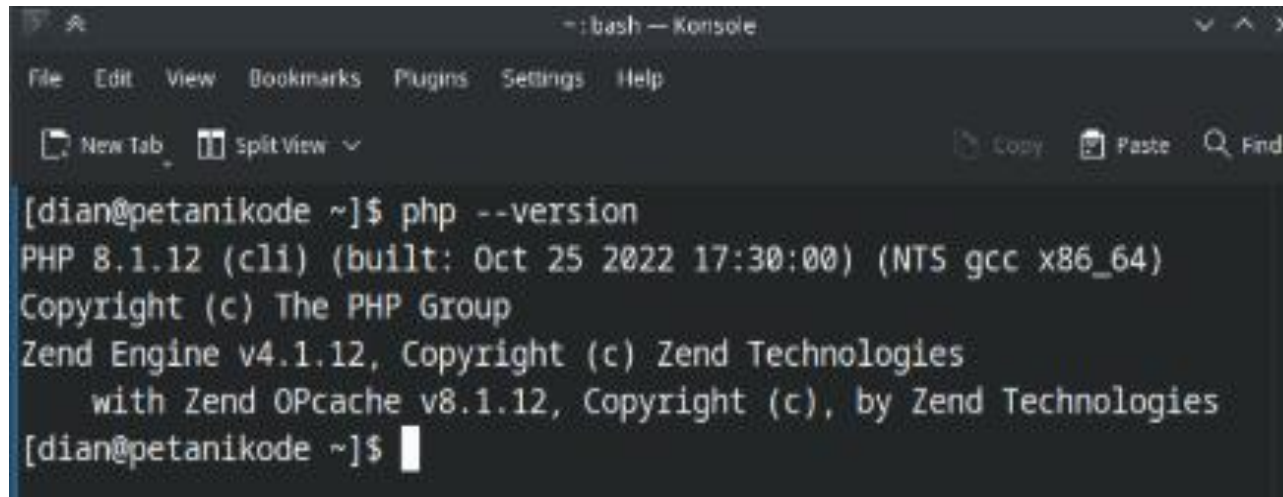
2. Browser

- Browser sudah pasti ada di setiap komputer.
- Ada banyak pilihan browser di Linux: Google Chrome, Firefox, Opera, dsb. Pilihlah browser yang kamu sukai.



3. PHP Interpreter

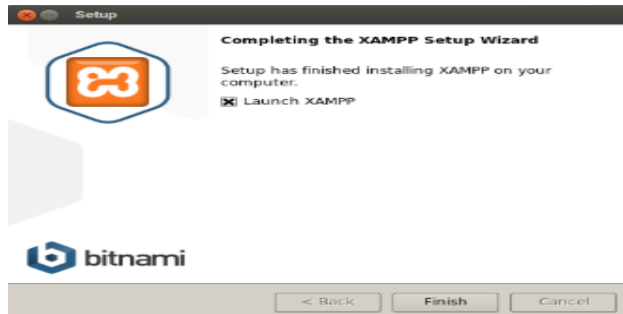
- Versi PHP terakhir saat ini adalah versi **8.1**, jadi pastikan kamu menginstall versi ini ya.
- Karena, masa dukungan PHP **8.1** sampai akhir tahun 2024.
- Jika kamu instal PHP 7, berarti kamu menggunakan PHP yang sudah usang.
- Coba ketik perintah: `php -v` `php --version`
- Hasilnya:



```
~: bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find
[dian@petanikode ~]$ php --version
PHP 8.1.12 (cli) (built: Oct 25 2022 17:30:00) (NTS gcc x86_64)
Copyright (c) The PHP Group
Zend Engine v4.1.12, Copyright (c) Zend Technologies
    with Zend OPcache v8.1.12, Copyright (c), by Zend Technologies
[dian@petanikode ~]$
```

4. Server Web (Apache)

- Sebenarnya PHP sudah membawa server web sendiri, jadi Apache sebenarnya tidak perlu kita install.
- XAMPP adalah sebuah paket aplikasi yang di dalamnya berisi webserver, PHP, MySQL , Phpmyadmin, perl, curl, dan program-program lainnya untuk mendukung pengembangan web.



- Berikutnya, coba buka browser dan buka alamat `http://localhost/`, maka akan tampil seperti ini.



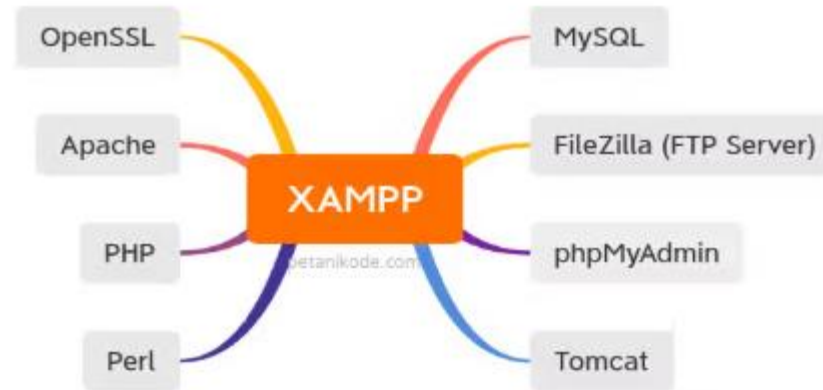
- XAMPP tidak ada di menu. Untuk membuka kembali XAMPP, kita harus mengeksekusi file `/opt/lampp/manager-linux-x64.run` sebagai **root**.

Mengenal Direktori htdocs

- Direktori **/opt/lampp/htdocs** adalah direktori tempat kita menaruh file-file web seperti **html** dan **php**. File ini nanti akan bisa diakses melalui alamat <http://localhost/>.
- Misalnya:
- Kita menaruh file **/opt/lampp/htdocs/webku.php**, maka cara membukanya di browser adalah dengan alamat <http://localhost/webku.php>.
- Namun, untuk saat ini, cuma **root** saja yang boleh menulis file di sana. Agar semua user bisa menulis di sana, kita harus memberikan hak akses.

XAMPP

- XAMPP adalah program untuk membuat server. XAMPP menyediakan paket server yang kita butuhkan untuk pengembangan web dengan PHP. Mulai dari PHP, MySQL, PHPmyadmin, dll.



• Cara Install XAMPP di Windows

XAMPP for Windows 7.2.32, 7.3.20 & 7.4.8

Version	Checksum	Size
7.2.32 / PHP 7.2.32	md5 sha1	154 Mb
7.3.20 / PHP 7.3.20	md5 sha1	154 Mb
7.4.8 / PHP 7.4.8	md5 sha1	155 Mb

Requirements Add-ons More Downloads »

Windows XP or 2003 are not supported. You can download a compatible version of XAMPP for these platforms [here](#).

Setup

Bitnami for XAMPP

Bitnami for XAMPP provides free installers that can install Drupal, Joomla!, WordPress and many other popular open source apps on top of your existing XAMPP installation.

<https://bitnami.com/xampp>

Learn more about Bitnami for XAMPP ☐

Tidak perlu dicentang

XAMPP Installer

< Back Next > Cancel

Setup

Completing the XAMPP Setup Wizard

Windows Security Alert

Windows Firewall has blocked some features of this app

Windows Firewall has blocked some features of Apache HTTP Server on all public and private networks.

Name: Apache HTTP Server

Publisher: Apache Software Foundation

Path: C:\xampp\apache\bin\httpd.exe

Allow Apache HTTP Server to communicate on these networks:

☒ Private networks, such as my home or work network

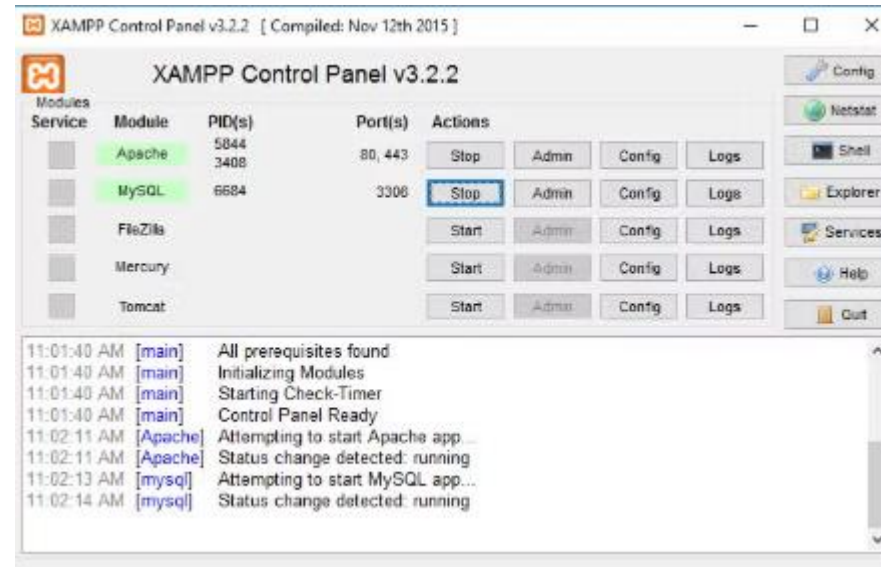
☐ Public networks, such as those in airports and coffee shops (not recommended because these networks often have little or no security)

[What are the risks of allowing an app through a firewall?](#)

Allow access Cancel

Manajemen Server XAMPP

- Pada Control Panel ini, kita bisa menyalakan dan mematikan, melakukan konfigurasi, memantau log, menjalankan shell, dan lain-lain.
- Untuk percobaan, coba klik tombol **Start** pada **Apache** dan **MySQL**. Sehingga statusnya akan menjadi *running*.



- Apache adalah webserver untuk menjalankan PHP dan MySQL adalah database server untuk menjalankan MySQL.
- Jika ada warna hijau dan ada nilai PID dan Port.. maka artinya server sudah berjalan. Tinggal kita pakai saja.

- Apa itu PID dan Port?
- **PID** adalah process ID, merupakan nomer proses yang dipakai pada task manager
- **Port** adalah nomer port yang akan dipakai untuk mengakses server.
- Nomer Port ini perlu kamu perhatikan, karena bisa jadi nomer port tersebut sudah dipakai oleh service yang lain.
- Biasanya apache akan menggunakan nomer port **80, 443** dan MySQL akan menggunakan nomer port **3306**.
- Untuk melihat service apa saja yang berjalan pada nomer port tertentu, silakan klik menu **Netstart**. Maka akan keluar seperti ini.

Netstat - TCP Listening sockets

Active socket New socket Old socket Refresh

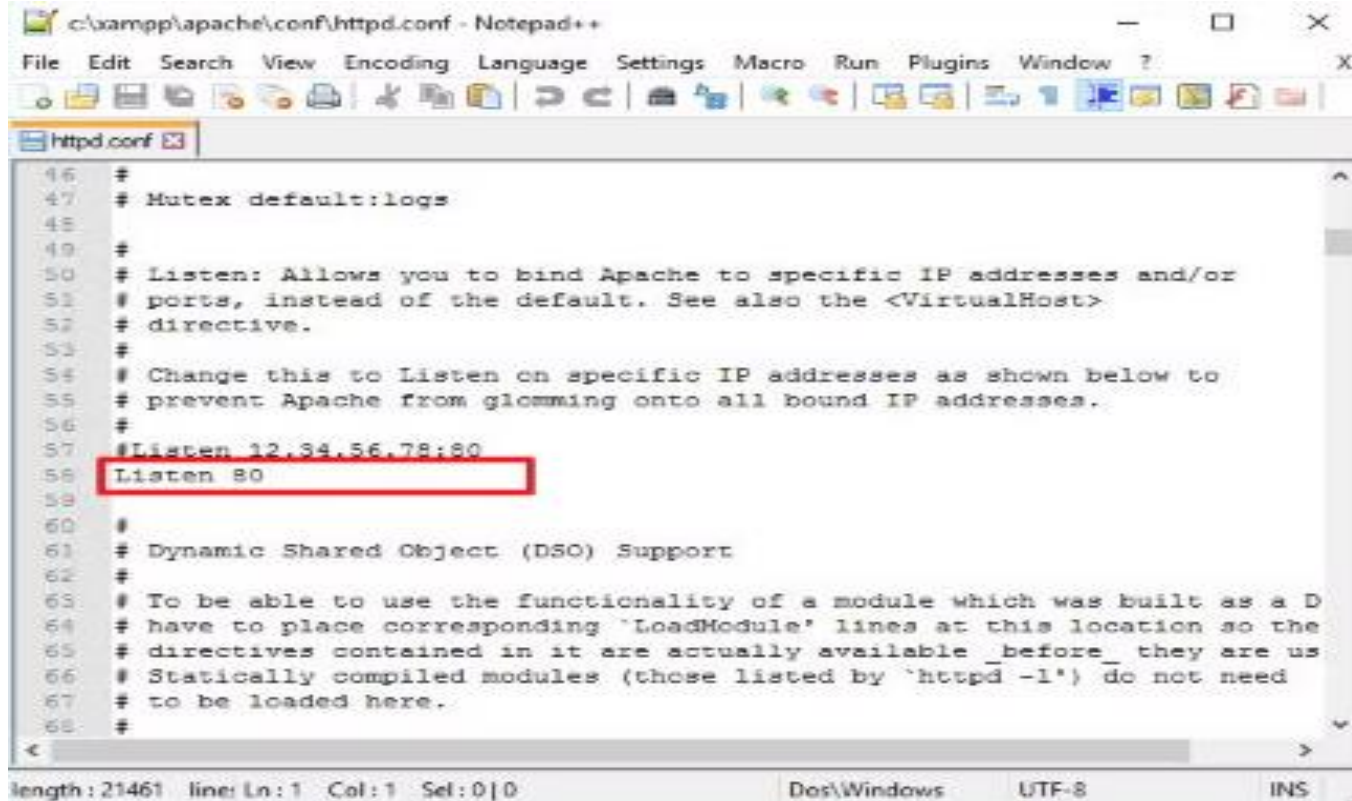
Address	Port	PID	Name
0.0.0.0	80	10020	httpd.exe
0.0.0.0	135	744	svchost.exe
0.0.0.0	443	10020	httpd.exe
0.0.0.0	445	4	System
0.0.0.0	554	5856	wmpnetwk.exe
0.0.0.0	2869	4	System
0.0.0.0	3306	9868	mysqld.exe
0.0.0.0	5357	4	System
0.0.0.0	10243	4	System
0.0.0.0	49064	480	wininit.exe
0.0.0.0	49065	956	svchost.exe
0.0.0.0	49066	1076	svchost.exe
0.0.0.0	49067	1584	spoolsv.exe
0.0.0.0	49068	600	services.exe
0.0.0.0	49075	608	lsass.exe
10.1.1.3	139	4	System
10.1.1.3	52093	4236	SkypeHost.exe
10.1.1.7	139	4	System
10.1.1.7	14959	4236	SkypeHost.exe
10.1.1.7	51249	1492	explorer.exe
10.1.1.7	51253	6796	OneDrive.exe
10.1.1.7	52097	4236	SkypeHost.exe
10.1.1.7	52132	4312	svchost.exe
10.1.1.7	52155	6796	OneDrive.exe
10.1.1.7	52156	6796	OneDrive.exe
10.1.1.7	52157	7100	MicrosoftEdgeCP.exe
10.1.1.7	52158	7100	MicrosoftEdgeCP.exe
10.1.1.7	52159	7100	MicrosoftEdgeCP.exe

- Dari menu ini, kita bisa memantau nomer port mana saja yang sudah terpakai. Sehingga nanti kalau kita mengalami port bentrok, kita bisa ganti dengan nomer port yang belum terpakai.
- Contohnya seperti ini:



- Server apache gagal dijalankan karena nomer port-nya sudah terpakai atau bentrok.
- Yang perlu kamu lakukan untuk mengatasi masalah ini adalah:
 1. Mematikan service yang berjalan di port tersebut;
 2. atau mengganti nomer port default dari Apache.
- Jika ingin mengganti nomer port default dari apache, caranya klik tombol **Config** di sebelah tombol **admin** di apache.

- Kemudian ubah **listen 80**, menjadi nomer port yang lain misalnya **listen 8080** maka nomer port apache akan menjadi **8080**.



The screenshot shows a Notepad++ window titled 'c:\xampp\apache\conf\httpd.conf - Notepad++'. The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Macro, Run, Plugins, Window, and Help. The toolbar contains various icons for file operations and editing. The active tab is 'httpd.conf'. The text content is as follows:

```
46 #
47 # Mutex default:logs
48
49 #
50 # Listen: Allows you to bind Apache to specific IP addresses and/or
51 # ports, instead of the default. See also the <VirtualHost>
52 # directive.
53 #
54 # Change this to Listen on specific IP addresses as shown below to
55 # prevent Apache from glomming onto all bound IP addresses.
56 #
57 #Listen 12.34.56.78:80
58 Listen 80
59
60 #
61 # Dynamic Shared Object (DSO) Support
62 #
63 # To be able to use the functionality of a module which was built as a D
64 # have to place corresponding 'LoadModule' lines at this location so the
65 # directives contained in it are actually available _before_ they are us
66 # Statically compiled modules (those listed by 'httpd -l') do not need
67 # to be loaded here.
68 #
```

The status bar at the bottom shows 'length: 21461 line: Ln: 1 Col: 1 Sel: 0|0', 'Dos/Windows' line endings, 'UTF-8' encoding, and 'INS' cursor type.

- Setelah itu simpan, dan coba restart server apache.

Syntaks Dasar PHP

- Struktur Program PHP yang Paling Dasar

```
<?php  
echo "Hello World!";  
?>
```

- Keterangan:
- **<?php** ini adalah pembuka program PHP. Pembuka ini wajib ada di setiap program PHP.
- **echo "Hello World!";** adalah sebuah statement atau perintah untuk menampilkan teks.
- **?>** Tutup program dibutuhkan saat kita menggabungkan kode PHP dengan HTML.
- Contoh :

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title><?php echo "Belajar PHP" ?></title>  
  </head>  
  <body>  
    <?php  
      echo "saya sedang belajar PHP<br>";  
      echo "<p>Belajar PHP hingga jadi master</p>";  
    ?>  
  </body>  
</html>
```

- Apa yang akan terjadi bila kita menghapus tutup PHP (=)? Tentunya program akan error.</li- PHP yang ditulis di dalam HTML, filenya harus disimpan dengan ekstensi **.php** bukan **.html** meskipun isinya HTML dan PHP.
- Lalu ada juga yang menulis seperti ini:

```
<?php  
  
echo "<html>";  
echo "<head>";  
echo "<title>Judul Web</title>";  
echo "</head>";  
echo "<body>";  
echo "<h1>Selamat datang</h1>";  
echo "</body>";  
echo "</html>";
```

- kalau yang ini bisa tidak ditutup, karena kode HTML-nya ditulis di dalam sintak PHP.

Penulisan Statement dan Ekspresi

- Statement dan ekspresi adalah instruksi yang akan diberikan ke komputer. Setiap statement dan ekspresi di PHP harus diakhiri dengan titik koma (;).
- Contoh:

```
<?php  
echo "ini statement 1";  
echo "ini statement 2";  
$a = $b + $c;
```
- Gimana kalau kita lupa menuliskan titik koma? Ya tentu programnya akan error.

Aturan Penulisan Case PHP

- PHP adalah bahasa pemrograman yang bersifat *case sensitive*. Artinya, huruf besar (kapital) dan huruf kecil akan dibedakan.

```
<?php  
  
$nama = "petanikode";  
$NAMA = "dian";  
  
?>
```

- Variabel **\$nama** dan **\$NAMA** adalah dua variabel yang berbeda. Mereka tidak sama.
- Penulisan huruf besar dan kecil dalam program harus diperhatikan, karena bisa menyebabkan error bila salah.
- sering menemukannya, banyak yang salah tulis.

```
$umur = 19;  
  
echo "Umur saya adalah $Umur";
```

- ada kode di atas, kita membuat variabel **\$umur** dengan huruf kecil. Lalu saat menggunakan variabel kita menggunakan **\$Umur**. Jelas sekali ini akan menyebabkan error. Untuk menghindari ini, kita harus konsisten dalam penamaan variabel dan konstanta.

- Gunakan nama variabel dengan huruf kecil saja dan konstanta dengan huruf kapital.

```
const INI_KONSTANTA = 123;  
$ini_variabel = 23;  
$iniJugaVariabel = 49;
```

- ada huruf kapital di **\$iniJugaVariabel** ?
- Ini namanya *camelCase*. Jika variabel terdiri dari dua atau lebih suku kata, maka kita bisa memisahkannya dengan huruf kapital atau bisa juga dengan garis bawah (*underscore*).

Penulisan Komentar di PHP

- Komentar adalah bagian yang tidak akan dieksekusi oleh komputer. Biasanya digunakan untuk keterangan, penjelasan, dan dokumentasi kode program.
- Komentar di PHP dapat ditulis dengan dua cara:
- Menggunakan tanda `//` untuk komentar satu baris;
- Menggunakan tanda `/*` untuk komentar lebih dari satu baris.

```
<?php

// ini adalah komentar
echo "Hello world";

/*
ini adalah komentar
yang lebih dari satu
baris
*/

?>
```

Penulisan Blok Program

- Blok program merupakan kumpulan dari statement dan ekspresi. Blok program di PHP dibungkus dengan kurung kurawal { ... }.

```
if ($umur > 18){  
    echo "Kamu sudah dewasa";  
    echo "Selamat datang";  
    echo "Kamu boleh minum kopi";  
}
```

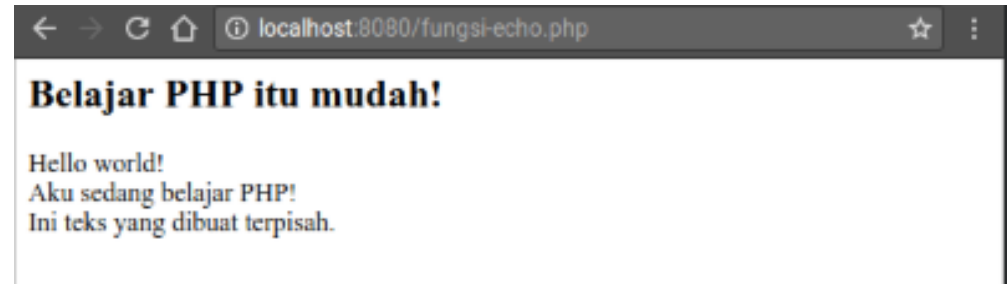
- Program di atas adalah contoh blok kode *if* yang berisi tiga statement.

- PHP memiliki beberapa fungsi untuk mencetak teks ke layar:
- Fungsi **echo()**;
- Fungsi **print()**;
- Fungsi **printf()**.

Fungsi echo()

- Fungsi **echo()** adalah fungsi untuk menampilkan teks ke layar. Fungsi ini dapat digunakan dengan tanda kurung maupun tanpa tanda kurung.

```
<?php
echo "<h2>Belajar PHP itu mudah!</h2>";
echo("Hello world!<br>");
echo "Aku sedang belajar PHP!<br>";
echo "Ini ", "teks ", "yang ", "dibuat ", "terpisah.";
?>
```

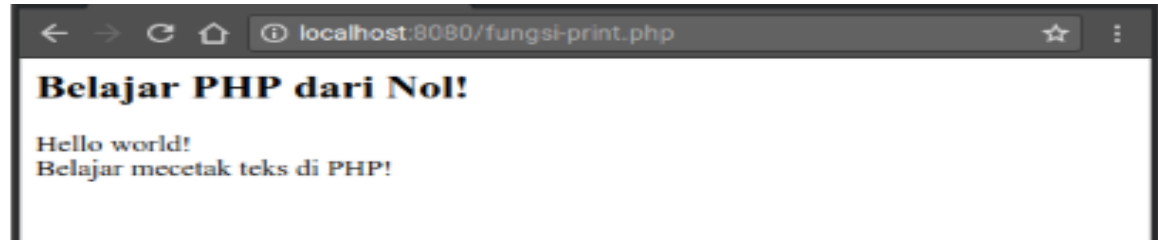


- Fungsi **echo()** tidak akan mengembalikan apa-apa setelah dieksekusi. Dia hanya bertugas menampilkan teks saja.

Fungsi print()

- Fungsi **print()** sama seperti fungsi **echo()**. Dia juga digunakan untuk menampilkan teks ke layar. Fungsi **print()** juga bisa digunakan tanpa tanda kurung.
- Perbedaannya dengan **echo()**:
- Fungsi **print()** akan selalu mengembalikan nilai **1** saat dieksekusi, sedangkan **echo()** tidak mengembalikan apa-apa.
- Fungsi **print()** hanya boleh diberikan satu parameter saja, sedangkan **echo()** boleh lebih dari satu.

```
<?php
print "<h2>Belajar PHP dari Nol!</h2>";
print "Hello world!<br>";
print "Belajar mencetak teks di PHP!";
?>
```



- Apabila kita membuat variabel lalu mengisinya dengan fungsi **print()** seperti ini:

```
$cetak = print("Hello World!");
```

- Maka variabel **\$cetak** akan bernilai 1. Apabila kita memberikan dua parameter ke dalam fungsi **print()**, maka akan terjadi error.

```
print("Hello", "World"); // <-- ini akan error
```

Menggabungkan String atau Teks di PHP

- Pada fungsi **echo()**, kita menggabungkan teks atau string dengan memberikan sebagai argumen (dipisah dengan tanda koma).

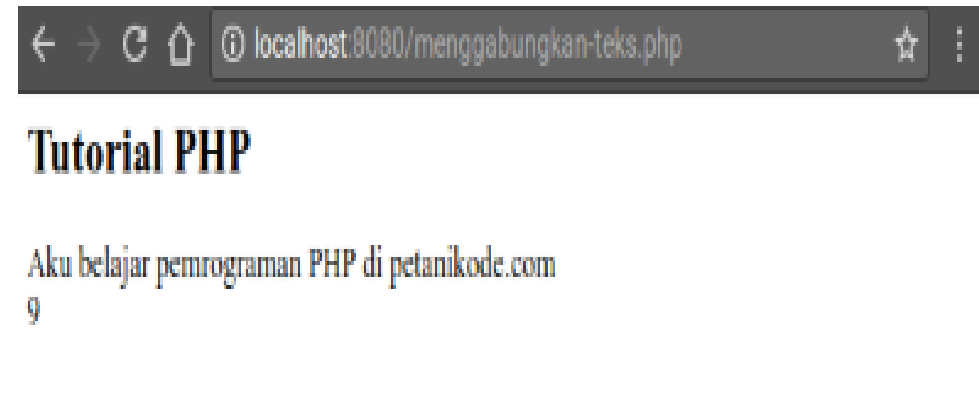
```
echo "Ini ", "teks ", "yang ", "dibuat ", "terpisah.";
```

- Selain cara ini, kita juga bisa melakukannya dengan tanda titik (.) Titik adalah operator untuk menggabungkan dua teks di PHP.
- Kenapa tidak menggunakan simbol plus (+)? Pada PHP, simbol plus digunakan untuk penjumlahan, bukan untuk menggabungkan string.

```
echo "1" + "1"; // akan menghasilkan 2  
echo "1" . "1"; // akan menghasilkan 11
```

- Contoh:

```
<?php  
$txt1 = "Tutorial PHP";  
$txt2 = "petanikode.com";  
$x = 5;  
$y = 4;  
  
echo "<h2>" . $txt1 . "</h2>";  
echo "Aku belajar pemrograman PHP di " . $txt2 . "<br>";  
echo $x + $y;  
?>
```



Fungsi printf() untuk Formatting Teks

- Fungsi **printf()** adalah fungsi untuk memformat teks atau string. Fungsi ini akan mengembalikan panjang dari teks saat dieksekusi.
- Biasanya saat kita menggunakan fungsi **echo**, kita akan menulis seperti ini:

```
$txt = "petanikode.com";  
echo "Aku belajar pemrograman PHP di " . $txt . "<br>";
```
- Apabila kita ingin menggunakan fungsi **printf()**, maka kita bisa lakukan seperti ini:

```
$txt = "petanikode.com";  
printf("Aku belajar pemrograman PHP di %s<br>", $txt);
```
- Simbol **%s** adalah sebuah *placeholder* untuk teks (string). Selain simbol **%s** ada juga simbol:
 - **%d** untuk bilangan desimal (integer);
 - **%f** untuk pecahan (float);
 - **%b** untuk boolean.
- Salah satu yang patut kita coba adalah **%f**, karena dengan ini kita bisa mengatur bagaimana bilangan pecahan ditampilkan.

- Contoh :

```
// misalkan kita punya bilangan dengan notasi E seperti ini


# $harga = 100000;



// jika kita cetak dengan echo:


# echo "Harganya adalah Rp $harga";



// jika kita cetak dengan printf


# printf("Harganya adalah Rp %.2f", $harga);


```

- Hasilnya :

```
Harganya adalah Rp 100000
Harganya adalah Rp 100000.00
```

- Simbol **%.2f** artinya kita akan menetak bilangan dengan dua angka di belakang koma.

Mencetak Tanda Kutip

- Kadang kita ingin menetak tanda kutip seperti ini:

```
echo 'hari ini hari jum'at';
```

- Maka akan terjadi error! Karena di sana dianggap tutup teksnya berakhir pada teks **jum.**
- Lalu bagaimana caranya kita mencetak tanda kutip? Kita bisa menggunakan tanda *back slash* (\).

```
echo 'hari ini hari jum\'at';
```

- Maka akan menghasilkan:

```
hari ini hari jum'at
```

Membuat Variabel di PHP

- Pada PHP, kita membuat variabel dengan tanda dolar (\$), lalu diikuti dengan nama variabelnya serta nilai yang ingin kita simpan.
- membuat variabel Bernama **\$harga** dengan isi **1000**.

```
<?php  
  
$harga = 1000;
```

- Tanda sama dengan (=) adalah simbol atau operator yang digunakan untuk mengisi nilai ke variabel.

```
<?php  
  
$nama_barang = "Kopi C++";  
$harga = 4000;  
$stok = 40;
```

- variabel juga dapat diisi ulang dengan nilai yang baru.

```
<?php  
  
// membuat variabel baru  
$stok = 40;  
  
// mengisi ulang variabel dengan nilai baru  
$stok = 34;
```

- Dalam membuat nama variabel ada beberapa hal yang harus diperhatikan:
- Awal dari nama variabel tidak boleh menggunakan angka dan simbol, kecuali underscore.

```
$!nama = "";  
$46rosi = "";
```

```
$nama = "";  
$rosi46 = "";
```

- Nama variabel yang terdiri dari dua suku kata, bisa dipisah dengan underscore (_) atau menggunakan style camelCase.

```
$nama_barang = "Buku PHP";  
$namaPembeli = "Petani Kode"; // <-- menggunakan camelCase
```

- Variabel harus diisi saat pembuatannya. Bila kita tidak ingin mengisi, cukup isi dengan nilai kosong.

```
$nama_barang = "";  
$namaPembeli = "";  
$harga = 0;
```

- Nama variabel bersifat *Case Sensitive*, artinya huruf besar dan huruf kecil dibedakan.

```
$belajar = "";  
$BELAJAR = "";  
$belajar = "";
```

Mengambil Nilai dari Variabel

- Mengambil nilai dari variabel bisa kita lakukan dengan menuliskan namanya dalam perintah **echo** maupun ekspresi yang lain.

```
<?php
// membuat variabel baru
$nama_barang = "Minyak Goreng";
$harga = 15000;

// menampilkan isi variabel
echo "Ibu membeli $nama_barang seharga Rp $harga";
```



- Jika kita menggunakan tanda petik ganda (") dalam **echo**, maka kita bisa menuliskan langsung nama variabelnya seperti ini:

```
$judul = "Belajar PHP dari nol sampai expert";
echo "Judul artikel: $judul";
```

- Apabila kita menggunakan tanda petik tunggal ('), maka kita harus menggunakan titik untuk menggabungkan teks dengan variabelnya.

```
$judul = "Tutorial PHP untuk Pemula";
echo 'Judul artikel: '.$judul;
```

- Tanda titik pada perintah **echo** berfungsi untuk menggabungkan teks yang ada di dalam variabel dengan teks yang akan kita cetak.

Mengenal Tipe Data di PHP

- Variabel yang sudah kita buat bisa kita simpan dengan berbagai jenis data. Jenis-jenis data ini disebut tipe data.
- Ada beberapa macam tipe data yang dapat disimpan dalam variabel:
 1. Tipe data char (karakter)
 2. Tipe data string (teks)
 3. Tipe data integer (angka)
 4. Tipe data float (pecahan)
 5. Tipe data boolean
 6. Tipe data objek
 7. Tipe data Array
 8. NULL
 9. dll.

- Pada PHP, kita tidak harus mendeklarasikan tipe datanya secara eksplisit. Karena PHP sudah mampu mengenali tipe data dari nilai yang kita berikan.

```
<?php

// tipe data char (karakter)
$jenis_kelamin = 'L';

// tipe data string (teks)
$nama_lengkap = "Petani Kode";

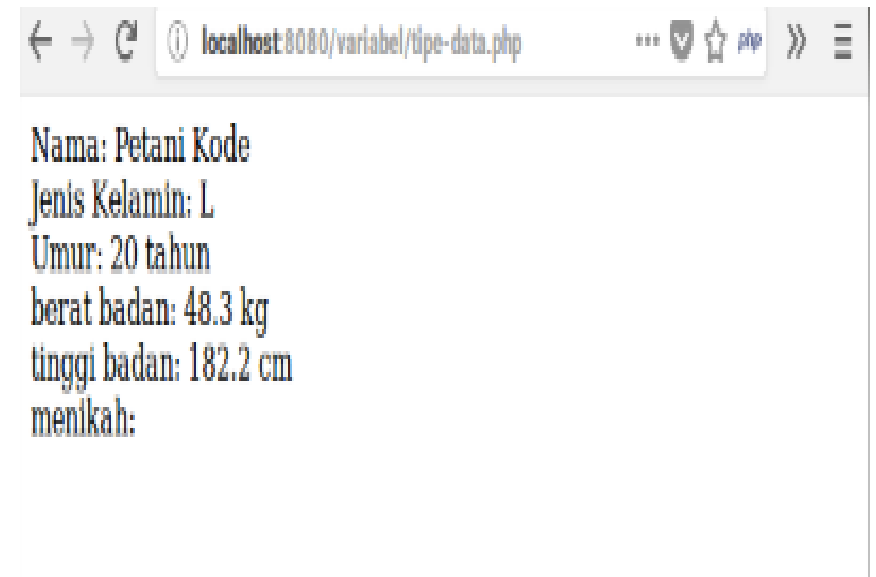
// tipe data integer
$umur = 20;

// tipe data float
$berat = 48.3;

// tipe data float
$tinggi = 182.2;

// tipe data boolean
$menikah = false;

echo "Nama: $nama_lengkap<br>";
echo "Jenis Kelamin: $jenis_kelamin<br>";
echo "Umur: $umur tahun<br>";
echo "berat badan: $berat kg<br>";
echo "tinggi badan: $tinggi cm<br>";
echo "menikah: $menikah";
```



- Variabel **\$menikah** akan ditampilkan kosong, karena nilai **false** akan dikonversi menjadi kosong dalam string.

- **Tipe Data Char dan String di PHP**

- Char adalah tipe data yang terdiri dari karakter. Penulisannya diapit dengan tanda petik satu.

```
$huruf = 'E';
```

- Lalu, *String* adalah tipe data yang terdiri dari kumpulan karakter. Penulisannya diapit dengan tanda petik ganda.

```
$alamat = "Jl. Mawar, Jakarta";
```

- **Tipe Data Integer di PHP**

- Integer adalah tipe data angka. Penulisannya tidak menggunakan tanda petik.

```
$nilai = 98; // angka positif  
$poin = -31; // angka negatif
```

- **Tipe Data Float di PHP**

- Float adalah tipe data bilangan pecahan. Sama seperti integer, tipe data ini ditulis tanpa tanda petik. tipe data float ditulis dalam notasi seperti ini:

```
$panjang = 233.12;  
$kapasitas = 13232.12;
```

```
$jarak = 1.2E-5;
```

- **E-5** artinya eksponen dari **10**. Contoh di atas akan sama dengan **1.2 x 10⁻⁵**. Kalau kita jabarkan akan menjadi **0.000012**. Agar format float tidak tercetak dalam notasi **E**, kita bisa gunakan fungsi **sprintf()**.

```
echo sprintf('%f', $a);  
// batasi angka di belakang koma  
echo sprintf('%.3f', $a);
```

- **Tipe data Boolean di PHP**

- Tipe data *boolean* adalah tipe data yang hanya bernilai **true** dan **false**. Penulisan **true** dan **false** tidak diapit dengan tanda petik.

```
$isActive = false;  
$menikah = true;
```

- **Tipe Data Array dalam PHP**

- Array adalah tipe data yang berisi sekumpulan data.

```
$minuman = array("Kopi", "Teh", "Jus Jeruk");  
$makanan = ["Nasi Goreng", "Soto", "Bubur"];
```

- **Tipe Data Objek di PHP**

- Tipe data objek adalah tipe data abstrak yang berisi data dan *method*.

```
$user = new User();
```

- Tipe data objek lebih sering disebut *instance* dari sebuah class. Pada contoh di atas **User()** adalah class yang di-instance di variable **\$user**.
- Setiap pembuatan *instance* harus didahului dengan kata kunci **new**.

- **Tipe Data NULL di PHP**

- Tipe data **NULL** adalah tipe data yang menyatakan kosong. Artinya: Jika kita mengisi variabel dengan nilai **NULL**, maka variabel tersebut akan dianggap kosong atau tidak punya nilai.

```
$nama = NULL;
```

- **Konversi Tipe data di PHP**

- Apa yang akan terjadi bila kita melakukan pembagian dengan tipe data teks dengan angka seperti ini:

```
$a = 3;  
$b = "angka 10";  
  
$c = $b / $a;
```

- Tentunya akan terjadi error saat kita ingin mencetak isi variabel **\$c**. Karena itu, kita harus konversi dulu tipe datanya agar bisa dilakukan operasi yang lain. Pada PHP konversi tipe data bisa dilakukan dengan operator kali (*).

```
<?php  
$foo = "1"; // mula-mula $foo dalam bentuk string (ASCII 49)  
$foo *= 2; // $foo sekarang adalah integer (2)  
$foo = $foo * 1.3; // $foo sekarang adalah float (2.6)  
$foo = 5 * "10 Little Piggies"; // $foo sekarang adalah integer (50)  
$foo = 5 * "10 Small Pigs"; // $foo sekarang adalah integer (50)  
?>
```

- Selain cara ini, kita juga bisa melakukannya seperti di bahasa C:

```
<?php  
  
$a = "32";  
$a = (int) $a; // ubah nilai a menjadi integer  
$a = (float) $a; // ubah nilai a menjadi float  
$a = (string) $a; // ubah nilai a menjadi string  
  
?>
```

- **Menghapus Variabel dari Memori**
- Apabila ada variabel yang sudah tidak dibutuhkan lagi, maka kita bisa menghapusnya untuk meningkatkan performa program.
- Cara menghapus variabel di PHP dapat menggunakan fungsi **unset()**.

```
// membuat variabel $tmp  
$tmp = 2901;  
  
// menghapus variabel $tmp  
unset($tmp);  
  
// mencoba mengakses variabel $tmp  
echo $tmp;
```

- Jika kita eksekusi kode di atas, maka akan terjadi error:

```
PHP Notice: Undefined variable: tmp
```

- Karena variabel **\$tmp** sudah kita hapus. Kenapa harus menghapus variabel? nanti juga akan terhapus sendiri saat programnya selesai?
- Biasanya saat kita ingin optimasi, variabel-variabel yang tidak terpakai harus dihapus agar tidak membebani memori pada server.

Konstanta

- Konstanta seperti variabel. Ia bisa menyimpan nilai. Tapi tidak bisa diubah.

```
// kita punya konstanta dengan nilai 5
const SEBUAH_NILAI = 5

// lalu kita ubah menjadi 10
SEBUAH_NILAI = 10 // <-- maka akan terjadi error di sini
```

- Kenapa bisa error? Karena sifat konstanta memang begitu. Nilai konstanta tidak bisa kita ubah-ubah seperti variabel.

- **Kapan Kita Harus Menggunakan Konstanta?**

- Konstanta biasanya digunakan untuk menyimpan nilai yang tidak pernah berubah.

```
const PHI = 3.14;  
const API_KEY = "182939812739812478u12ehj1h2u3123h12";
```

- Pada dunia nyata, nilai **PHI** memang tidak akan pernah berubah. Lalu untuk **API_KEY** biasanya kita gunakan untuk mengakses sebuah web service dan nilai ini tidak akan pernah berubah di dalam program.
- Intinya:
- Jika kita ingin menyimpan nilai yang tak akan pernah berubah, maka gunakanlah konstanta.
- Sedangkan apabila nilai tersebut berubah-ubah di dalam program, maka gunakanlah variabel.

- **Cara Membuat Konstanta dan Contohnya**
- Pada PHP, kita dapat membuat konstanta dengan dua cara.
- 1. Menggunakan fungsi **define()**;
- 2. Menggunakan kata kunci **const**.

```
<?php
// membuat konstanta dengan fungsi define()
define('DB_SERVER', 'localhost');
define('DB_USER', 'petanikode');
define('DB_PASS', 'R4Hasia');
define('DB_NAME', 'belajar');

// membuat konstanta dengan kata kunci const
const API_KEY = "1234";
?>
```

- Nama konstanta diharuskan menggunakan huruf kapital agar mudah dibedakan dengan variabel. Meskipun di PHP menggunakan simbol dolar (\$) untuk variabel, kita harus mengikuti aturan ini.

nama Konstanta Nilai yang disimpan

```
define('DB_HOST', 'localhost');
const DB_NAME = "petanikode";
```

nama Konstanta Nilai yang disimpan

- Lalu, bagaimana cara mengambil nilai dari konstanta? Sama seperti variabel, kita tinggal tulis namanya.

- Perhatikan!

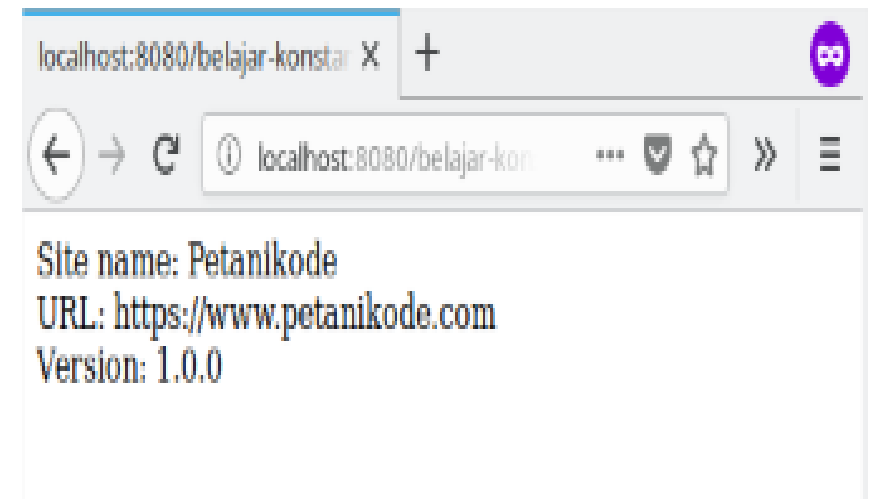
```
const SITE_NAME = "Petani Kode";  
  
echo "Nama situs: " . SITE_NAME;
```

- Kita menggunakan titik (.) untuk menggabungkan dua buah string. Karena konstanta tidak menggunakan dolar, kita tidak bisa langsung menulisnya seperti ini:

```
echo "Nama situs: SITE_NAME";
```

- Biar makin mantap, coba contoh program berikut:

```
<?php // file: belajar-konstanta.php  
  
// membuat konstanta  
define('VERSION', '1.0.0');  
  
const SITE_NAME = "Petani Kode";  
const BASE_URL = "https://www.petanikode.com";  
  
// cetak nilai konstanta  
echo "Site name: " . SITE_NAME . "<br/>";  
echo "URL: " . BASE_URL . "<br/>";  
echo "Version: " . VERSION . "<br/>";
```



Operator dalam PHP

- Operator adalah simbol-simbol yang digunakan untuk melakukan operasi terhadap suatu nilai dan variabel.
- Ada 6 Jenis operator dalam pemrograman PHP yang harus kita ketahui:
 - 1.Operator Aritmatika;
 - 2.Operator Penugasan atau *Assignment*;
 - 3.Operator *Increment & Decrement*;
 - 4.Operator Relasi atau pembandingan;
 - 5.Operator Logika;
 - 6.Operator Bitwise;
 - 7.dan Operator Ternary.

Operator Aritmatika

- Operator aritmatika merupakan operator untuk melakukan operasi aritmatika. Operator aritmatika terdiri dari:

Nama Operator	Simbol
Penjumlahan	+
Pengurangan	-
Perkalian	*
Pemangkatan	**
Pembagian	/
Sisa Bagi	%

- Contoh:

```
<?php

$a = 5;
$b = 2;

// penjumlahan
$c = $a + $b;
echo "$a + $b = $c";
echo "<hr>";

// pengurangan
$c = $a - $b;
echo "$a - $b = $c";
echo "<hr>";

// Perkalian
$c = $a * $b;
echo "$a * $b = $c";
echo "<hr>";

// Pembagian
$c = $a / $b;
echo "$a / $b = $c";
echo "<hr>";

// Sisa bagi
$c = $a % $b;
echo "$a % $b = $c";
echo "<hr>";

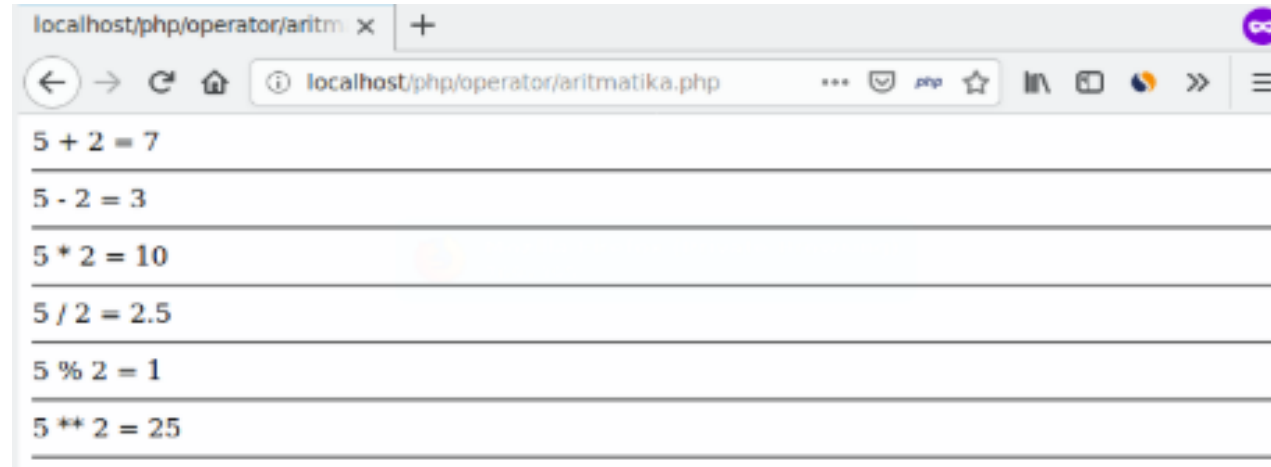
// Pangkat
$c = $a ** $b;
echo "$a ** $b = $c";
echo "<hr>";

?>
```

- Mula-mula kita punya dua variabel, yaitu **\$a** dan **\$b** dengan nilai awal sebagai berikut:

```
$a = 5;
$b = 2;
```

- Kemudian kita menggunakan operator aritmatika untuk melakukan operasi terhadap dua nilai atau variabel tersebut.
- Lalu hasilnya disimpan di dalam variabel **\$c**.
- Maka hasilnya:

A screenshot of a web browser window. The address bar shows 'localhost/php/operator/aritmatika.php'. The page content displays seven arithmetic operations, each on a new line and separated by a horizontal line: '5 + 2 = 7', '5 - 2 = 3', '5 * 2 = 10', '5 / 2 = 2.5', '5 % 2 = 1', and '5 ** 2 = 25'.

```
5 + 2 = 7
5 - 2 = 3
5 * 2 = 10
5 / 2 = 2.5
5 % 2 = 1
5 ** 2 = 25
```

- perhatikan juga simbol-simbol operator yang dipakai.
- Pada matematika, perkalian biasanya menggunakan simbol **x** Namun di dalam pemrograman perkalian menggunakan simbol *****.

```
// salah
$c = 7 x 4;

// benar
$c = 7 * 4;
```

- Lalu untuk operator **%** (modulo), ini adalah operator untuk menghitung sisa bagi.

- Misalnya:

```
// kita punya 5
$a = 5;

// lalu kita bagi dengan 2
$c = $a / 2;
```

- Maka variable **\$c** akan bernilai **1**, karena **5** dibagi **2** sisanya **1**.
- Biar lebih mudah, coba bayangkan seperti ini:
- Kamu punya permen **5** biji, lalu dibagi berdua dengan adikmu. Biar adil, sama-sama dapat dua biji. Nah, ada sisanya satu yang belum dibagi. Sisa inilah yang menjadi hasil modulo.

Operator Penugasan

- Operator ini adalah operator untuk memberikan tugas kepada variabel. Biasanya digunakan untuk mengisi nilai.
- Sama dengan (=) adalah operator penugasan untuk mengisi nilai. Selain sama dengan, terdapat juga beberapa operator penugasan seperti:

Nama Operator	Sombol
Pengisian Nilai	=
Pengisian dan Penambahan	+=
Pengisian dan Pengurangan	-=
Pengisian dan Perkalian	*=
Pengisian dan Pemangkatan	**=
Pengisian dan Pembagian	/=
Pengisian dan Sisa bagi	%=
Pengisian dan Penggabungan (string)	.=

- Apa bedanya dengan operator aritmatika?
- Operator penugasan digunakan untuk mengisi nilai dan juga menghitung dengan operasi aritmatika. Sedangkan operator aritmatika hanya berfungsi untuk menghitung saja.

- Sebagai contoh:

```
$speed = 83;

// ini operator aritmatika
$speed = $speed + 10;

// maka nilai speed akan sama dengan 83 + 10 = 93

// ini operator penugasan
$speed += 10;

// sekarang nilai speed akan menjadi 93 + 10 = 103
```

- Perhatikan operasi ini:

```
$speed = $speed + 10;
// dan
$speed += 10;
```

- Kedua operasi tersebut merupakan operasi yang sama. Hanya saja yang atas menggunakan operator aritmatika dan yang bawah menggunakan operator penugasan.
- Bisa dibilang, operator penugasan adalah bentuk yang lebih sederhana dari ekspresi seperti di atas. Penggunaan operator penugasan akan sering kita temukan saat membuat program.

- **Operator Increment & Decrement**

- Operator *increment* dan *decrement* merupakan operator yang digunakan untuk menambah **+1** (tambah satu) dan mengurangi **-1** (kurangi dengan satu).
- Operator *increment* menggunakan symbol **++**, sedangkan *decrement* menggunakan symbol **--**.

```
$score = 0;  
  
$score++;  
$score++;  
$score++;  
  
echo $score;
```

- Nilai **\$score** akan menjadi **3**, karena kita melakukan *increment* sebanyak 3x.

- **Operator Relasi**

- Operator relasi adalah operator untuk membandingkan dua buah nilai.
- Hasil operasi dari operator relasi akan menghasilkan nilai dengan tipe data *boolean*, yaitu **true** (benar) dan **false** (salah).
- Berikut ini daftar operator relasi:

Nama Operator	Simbol
Lebih Besar	>
Lebih Kecil	<
Sama Dengan	== atau ===
Tidak Sama dengan	!= atau !==
Lebih Besar Sama dengan	>=
Lebih Kecil Sama dengan	<=

- Mari kita coba dalam program:

```
<?php
$a = 6;
$b = 2;

// menggunakan operator relasi lebih besar
$c = $a > $b;
echo "$a > $b: $c";
echo "<hr>";

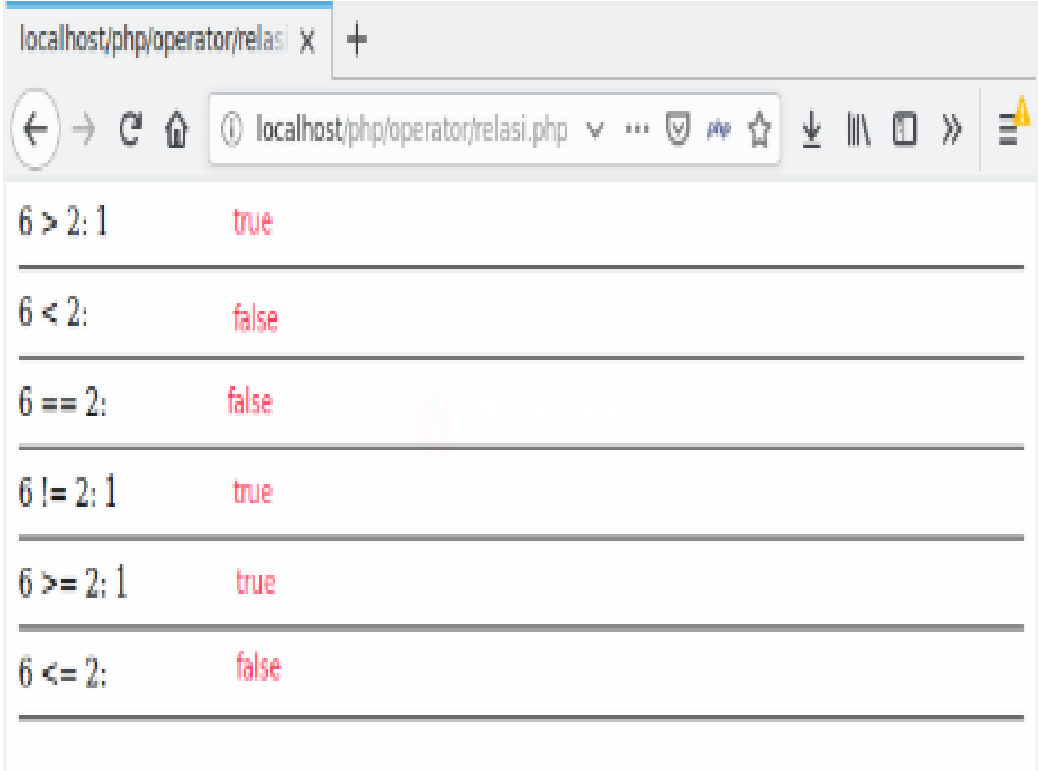
// menggunakan operator relasi lebih kecil
$c = $a < $b;
echo "$a < $b: $c";
echo "<hr>";

// menggunakan operator relasi lebih sama dengan
$c = $a == $b;
echo "$a == $b: $c";
echo "<hr>";

// menggunakan operator relasi lebih tidak sama dengan
$c = $a != $b;
echo "$a != $b: $c";
echo "<hr>";

// menggunakan operator relasi lebih besar sama dengan
$c = $a >= $b;
echo "$a >= $b: $c";
echo "<hr>";

// menggunakan operator relasi lebih kecil sama dengan
$c = $a <= $b;
echo "$a <= $b: $c";
echo "<hr>";
```



6 > 2: 1	true
6 < 2:	false
6 == 2:	false
6 != 2: 1	true
6 >= 2: 1	true
6 <= 2:	false

- Perhatikan!
- Di sana kita mendapatkan nilai **1** untuk **true** sedangkan **false** tidak ditampilkan atau **0**.
- Apakah ini salah? Tidak, memang seperti itulah sifat dari fungsi **echo** di PHP.
- Nilai dengan tipe data boolean biasanya tidak untuk ditampilkan. Biasanya digunakan untuk [pembuatan kondisi pada percabangan](#).
- Contohnya seperti ini:

```
<?php
$total_belanja = 150000;

if($total_belanja > 100000){
    echo "Anda dapat hadiah!";
}
```

- **Operator Logika**
- Jika kamu pernah belajar logika matematika, kamu pasti tidak akan asing dengan operator ini.
- Operator logika adalah operator untuk melakukan operasi logika seperti **AND, OR** dan **NOT**.
- Operator logika terdiri dari:

Nama Operator	Simbol
Logika AND	&&
Logika OR	
Negasi/kebalikan/ NOT	!

- Mari kita coba dalam program: **logika.php**

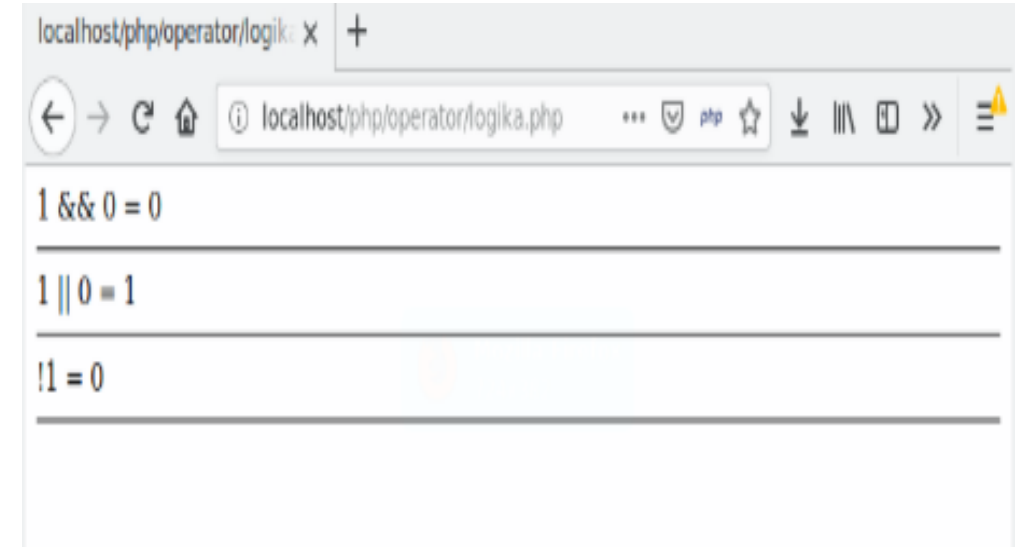
```
<?php

$a = true;
$b = false;

// variabel $c akan bernilai false
$c = $a && $b;
printf("%b && %b = %b", $a,$b,$c);
echo "<hr>";

// variabel $c akan bernilai true
$c = $a || $b;
printf("%b || %b = %b", $a,$b,$c);
echo "<hr>";

// variabel $c akan bernilai false
$c = !$a;
printf("!%b = %b", $a, $c);
echo "<hr>";
```



- Pada contoh di atas, kita menggunakan fungsi **printf()** untuk mencetak memformat dan mencetak teks.
- Namun akan tetap menampilkan **1** untuk **true** dan **0** untuk **false**.
- Operator logika sama seperti operator relasi, ia akan menghasilkan nilai dengan tipe data *boolean*.
- Perhatikanlah hasil yang di dapatkan ketika menggunakan operator **&** (AND), **||** (OR), dan **!** (NOT).

- Operator **&&** akan menghasilkan **true** apabila nilai kiri dan kanan bernilai **true**, Sedangkan operator **||** akan menghasilkan **false** saat nilai kiri dan kanan bernilai **false**.
- Coba cek kembali hukum logika AND, OR, dan NOT.

AND		Hasil
true	true	true
true	false	false
false	true	false
false	false	false

OR		Hasil
true	true	true
true	false	true
false	true	true
false	false	false

NOT	Hasil
true	false
false	true

- **Operator Bitwise**

- Operator bitwise merupakan operator yang digunakan untuk operasi bit (biner). Operator ini terdiri dari:

Nama	Simbol di Java
AND	&
OR	
XOR	^
Negasi/kebalikan	~
Left Shift	<<
Right Shift	>>

- Operator ini berlaku untuk tipe data **int**, **long**, **short**, **char**, dan **byte**.
- Operator ini akan menghitung dari bit-ke-bit.
- Misalnya, kita punya variabel **a = 60** dan **b = 13**.
- Bila dibuat dalam bentuk biner, akan menjadi seperti ini:

```
a = 00111100  
b = 00001101
```

- Kemudian, dilakukan operasi bitwise

- Operasi AND

```
a = 00111100  
b = 00001101  
a & b = 00001100
```

- Operasi OR

```
a = 00111100  
b = 00001101  
a | b = 00111101
```

- Operasi XOR

```
a = 00111100  
b = 00001101  
a ^ b = 00110001
```

- Operasi NOT (Negasi/kebalikan)

```
a = 00111100  
~a = 11000011
```

- Konsepnya memang hampir sama dengan operator Logika. Bedanya, Bitwise digunakan untuk biner.

- Untuk lebih jelasnya mari kita coba dalam program.

```
<?php

$a = 60;
$b = 13;

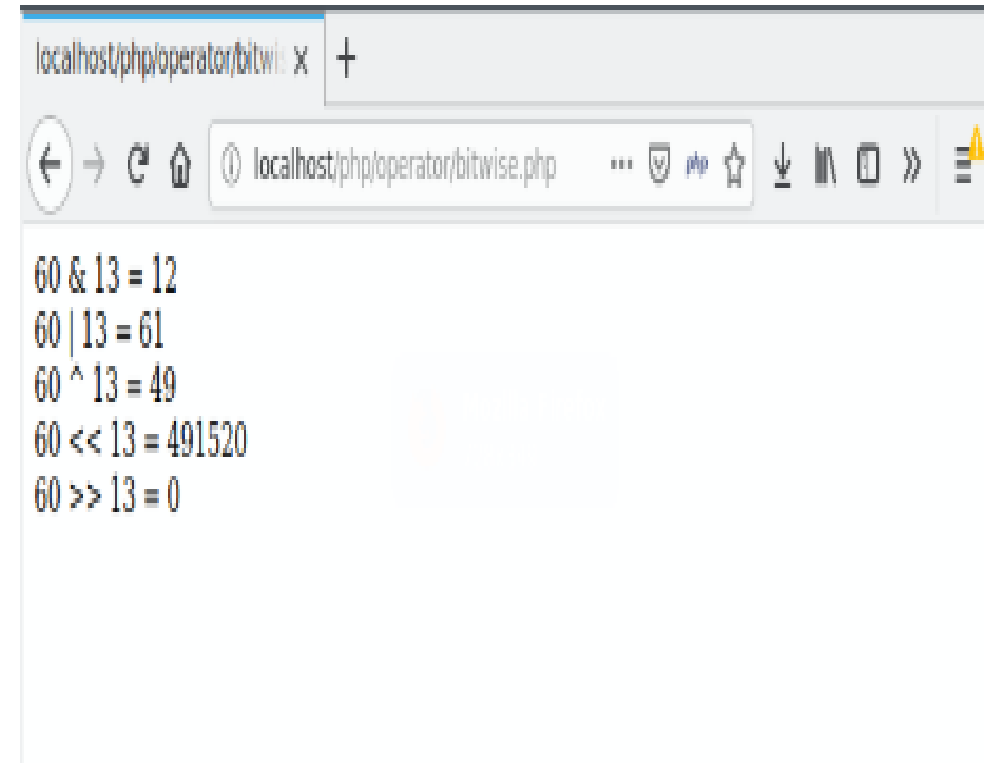
// bitwise AND
$c = $a & $b;
echo "$a & $b = $c";
echo "<br>";

// bitwise OR
$c = $a | $b;
echo "$a | $b = $c";
echo "<br>";

// bitwise XOR
$c = $a ^ $b;
echo "$a ^ $b = $c";
echo "<br>";

// Shift Left
$c = $a << $b;
echo "$a << $b = $c";
echo "<br>";

// Shift Right
$c = $a >> $b;
echo "$a >> $b = $c";
echo "<br>";
```



Operator Ternary

- Operator ternary adalah operator untuk membuat sebuah kondisi. Simbol yang digunakan adalah tanda tanya (?) dan titik dua (:).

Operator Ternary

kamu suka aku ? ya : tidak;

jawaban benar jawaban salah

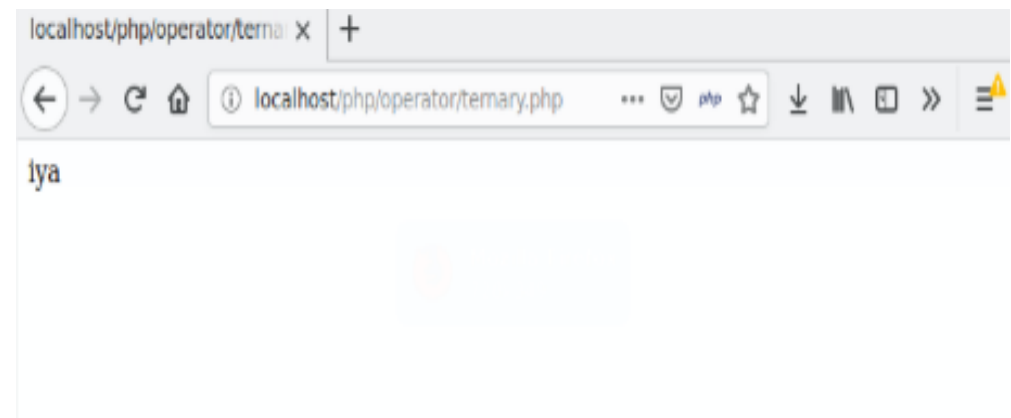
A diagram illustrating the ternary operator. The text "kamu suka aku ? ya : tidak;" is shown. Above the question mark is the label "Operator Ternary" with an arrow pointing to it. Below the question mark is the label "jawaban benar" with an arrow pointing to it. Below the colon and "tidak" is the label "jawaban salah" with an arrow pointing to it.

- Pada contoh di atas, “Kamu suka aku” adalah pertanyaan atau kondisi yang akan diperiksa. Kalau jawabannya benar, maka iya. Sebaliknya akan tidak. Untuk lebih jelasnya, mari kita coba...

```
<?php
$suka = true;

// menggunakan operator ternary
$jawab = $suka ? "iya" : "tidak";

// menampilkan jawaban
echo $jawab;
```



- Cobalah untuk mengganti nilai variabel **\$suka** menjadi **false**, maka hasil outputnya akan **tidak**.

Percabangan

- Percabangan adalah sebutan untuk alur program yang bercabang.



- Apabila kita ingin menambahkan percabangan, kita akan membuatnya seperti ini:



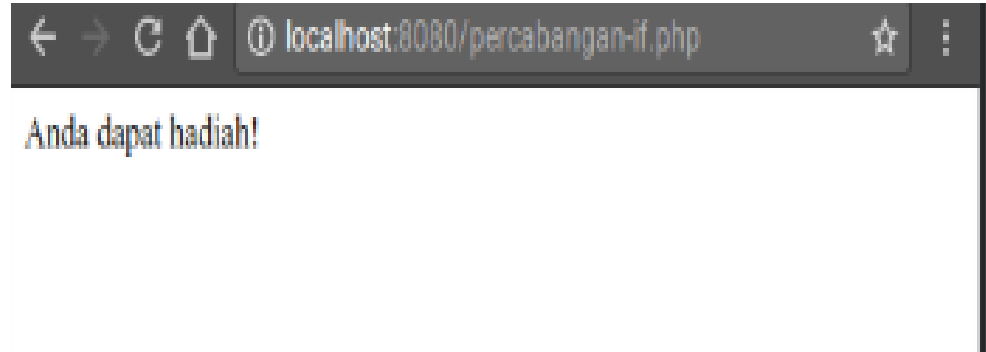
- **Percabangan If**

- Bentuk yang paling sederhana dari percabangan adalah “If” saja.
- Biasanya digunakan saat hanya ada satu tindakan yang harus dilakukan.
- Bentuknya seperti ini:

```
<?php  
  
if (<kondisi>){  
    // eksekusi kode ini  
}
```

- Jika kondisi benar, maka eksekusi kode yang ada di dalamnya. **<kondisi>** bisa kita isi dengan nilai *boolean* atau kita bisa buat pernyataan untuk menghasilkan nilai *boolean*.

```
<?php  
$total_belanja = 150000;  
  
if($total_belanja > 100000){  
    echo "Anda dapat hadiah!";  
}
```



- Perhatikan contoh di atas! Teks **Anda dapat hadiah!** hanya akan ditampilkan saat kondisi variabel **\$total_belanja** bernilai di atas **100000**. Kalau di bawah **100000**, tidak akan menampilkan apa-apa.

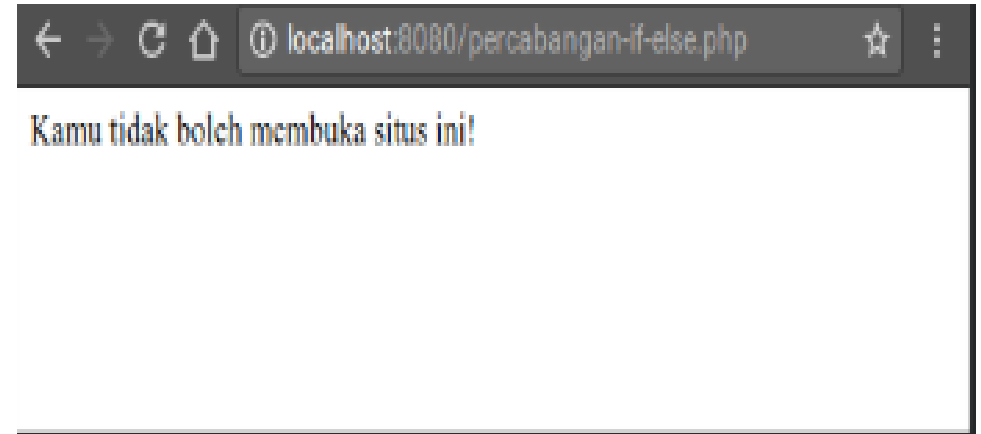
- Kondisi yang digunakan pada contoh adalah:

```
$total_belanja > 100000
```

- Kondisi atau pernyataan ini akan bernilai **true** dan **false**. Jika **true** (benar), maka kode yang ada di dalamnya akan dieksekusi. Namun, apabila **false** maka tidak akan mengeksekusinya.

- Percabangan If/Else
- Percabangan If/Else memiliki dua pilihan. Jika <kondisi> bernilai **false**, maka blok else akan dikerjakan.

```
<?php  
  
$umur = 13;  
  
if ($umur < 18 ){  
    echo "Kamu tidak boleh membuka situs ini!";  
} else {  
    echo "Selamat datang di website kami!";  
}  
?>
```



- Sekarang coba ubah nilai \$umur menjadi 19

```
<?php  
  
$umur = 19;  
  
if ($umur < 18 ){  
    echo "Kamu tidak boleh membuka situs ini!";  
} else {  
    echo "Selamat datang di website kami!";  
}  
?>
```



- Percabangan If/Elseif/Else
- Percabangan If/Elseif/Else memiliki lebih dari dua pilihan kondisi.

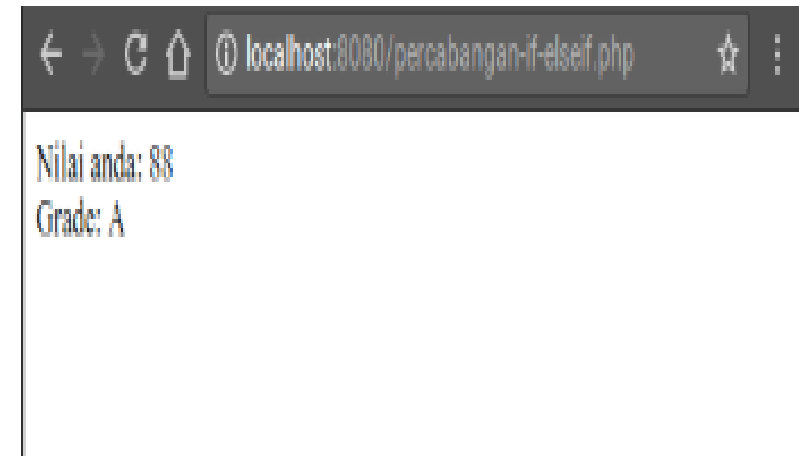
```
<?php

$nilai = 88;

if ($nilai > 90) {
    $grade = "A+";
} elseif($nilai > 80){
    $grade = "A";
} elseif($nilai > 70){
    $grade = "B+";
} elseif($nilai > 60){
    $grade = "B";
} elseif($nilai > 50){
    $grade = "C+";
} elseif($nilai > 40){
    $grade = "C";
} elseif($nilai > 30){
    $grade = "D";
} elseif($nilai > 20){
    $grade = "E";
} else {
    $grade = "F";
}

echo "Nilai anda: $nilai<br>";
echo "Grade: $grade";

?>
```



- Coba ubah variabel **\$nilai** menjadi **54** dan perhatikanlah hasilnya!

- **Percabangan Switch/Case**

- Percabangan Switch/Case adalah bentuk lain dari percabangan If/Elseif/Else.
- Format penulisannya seperti ini:

```
<?php

switch($variabel){
    case <konidisi>:
        // eksekusi kode ini
        break;
    case <kondisi2>:
        // eksekusi kode ini
        break;
    default:
        // eksekusi kode ini
}

?>
```

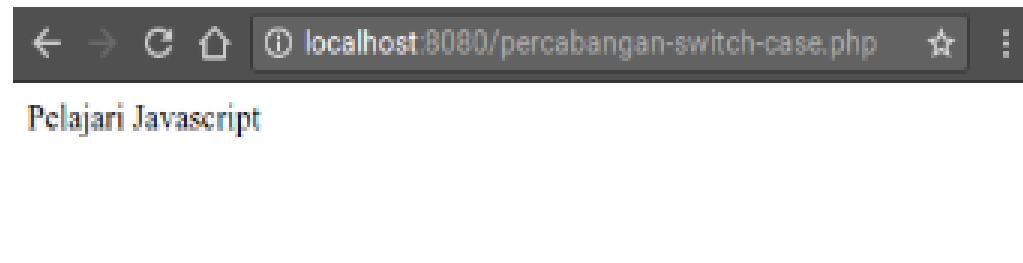
```
<?php

$level = 3;

switch($level){
    case 1:
        echo "Pelajari HTML";
        break;
    case 2:
        echo "Pelajari CSS";
        break;
    case 3:
        echo "Pelajari Javascript";
        break;
    case 4:
        echo "Pelajari PHP";
        break;
    default:
        echo "Kamu bukan programmer!";
}

?>
```

- Ada 5 pilihan dalam kondisi di atas. Pilihan **default** akan dipilih apabila nilai variable **\$level** tidak ada dalam pilihan **case**.



- Coba ubah nilai variable **\$level** dan perhatikanlah hasilnya.

- **Percabangan dengan Operator Ternary**

- Percabangan menggunakan operator ternary adalah bentuk sederhana dari percabangan If/Else.

- Formatnya seperti ini:

```
<?php
<kondisi> ? benar : salah;
?>
```

- Contoh:

```
<?php
$suka = true;

$suka ? echo "Aku juga suka kamu": echo "Baiklah!";

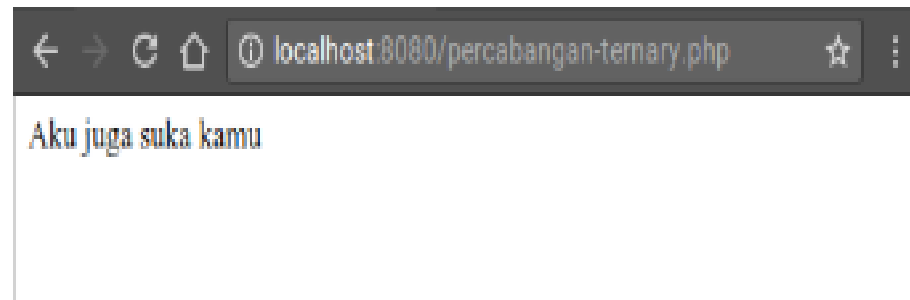
?>
```

```
<?php
$suka = true;

echo $suka ? "Aku juga suka kamu": "Baiklah!";

?>
```

- Artinya: jika variable **\$suka** bernilai **true** maka cetak "**Aku juga suka kamu**". Tapi kalau bernilai **false**, maka cetak "**Baiklah!**"



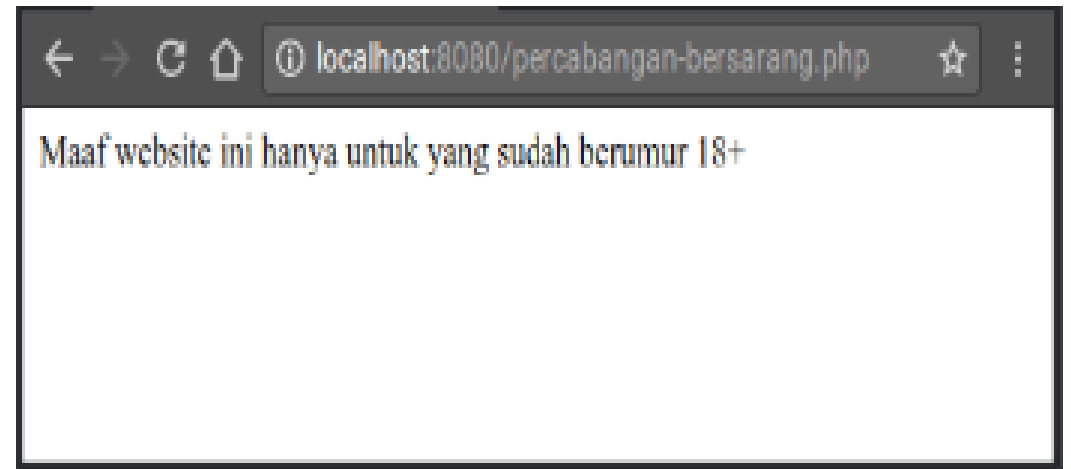
- **Percabangan Bersarang**
- Percabangan bersarang artinya ada percabangan di dalam percabangan (*nested*).

```
<?php

$umur = 17;
$menikah = false;

if($umur > 18){
    if($menikah){
        echo "Selamat datang pak!";
    } else {
        echo "Selamat datang wahai pemuda!";
    }
} else {
    echo "Maaf website ini hanya untuk yang sudah berumur 18+";
}

?>
```



Perulangan

- Ada dua jenis perulangan dalam pemrograman:
 - 1. Counted loop;
 - 2. Uncounted loop.
- *Counted loop* adalah perulangan yang sudah jelas banyak pengulangannya. Sedangkan Uncounted loop tidak pasti berapa kali dia akan mengulang.
- Pada PHP ada 4 jenis perulangan yang bisa kita gunakan:
 1. Perulangan For
 2. Perulangan While
 3. Perulangan Do/While
 4. Perulangan Foreach

- **Perulangan For**

- Perulangan *For* adalah perulangan yang termasuk dalam *counted loop*, karena kita bisa menentukan jumlah perulangannya.

- Bentuk dasar perulangan for:

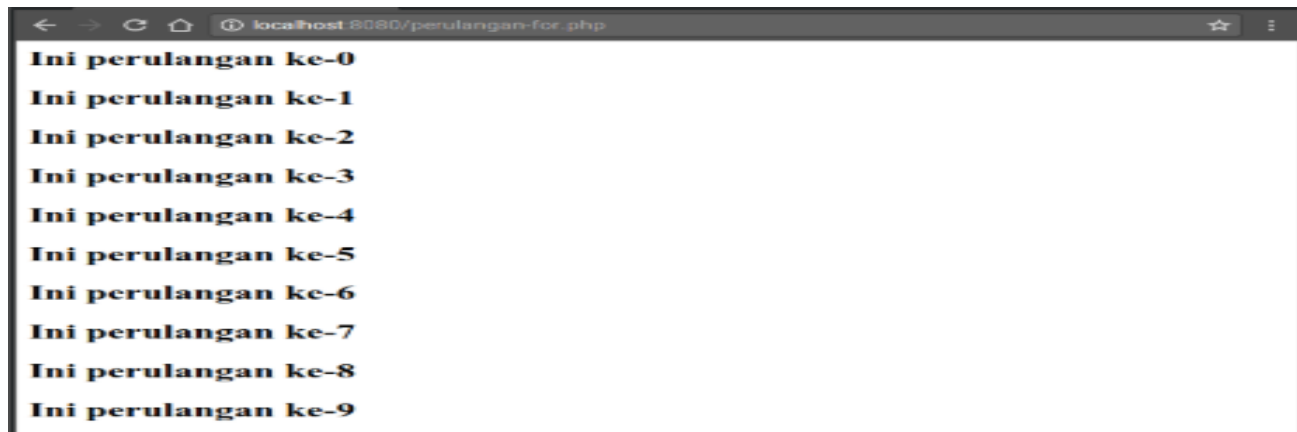
```
<?php
for ($i = 0; $i < 10; $i++){
    // blok kode yang akan diulang di sini!
}

?>
```

- Variabel **\$i** dalam perulangan *For* berfungsi sebagai *counter* yang menghitung berapa kali ia akan mengulang.
- Hitungan akan dimulai dari nol (**0**), karena kita memberikan nilai **\$i = 0**. Lalu, perulangan akan diulang selama nilai **\$i** lebih kecil dari **10**. Artinya, perulangan ini akan mengulang sebanyak **10x**.
- Maksud dari **\$i++** adalah nilai **\$i** akan ditambah **1** di setiap kali melakukan perulangan.

```
<?php
for($i = 0; $i < 10; $i++){
    echo "<h2>Ini perulangan ke-$i</h2>";
}

?>
```



```
Ini perulangan ke-0
Ini perulangan ke-1
Ini perulangan ke-2
Ini perulangan ke-3
Ini perulangan ke-4
Ini perulangan ke-5
Ini perulangan ke-6
Ini perulangan ke-7
Ini perulangan ke-8
Ini perulangan ke-9
```

- **Perulangan Foreach**

- Perulangan *foreach* sama seperti perulangan *for*. Namun, ia lebih khusus digunakan untuk mencetak array.
- Bentuk perulangan *foreach*:

```
<?php

foreach($array as $data){
    echo $data;
}
```

- Contoh:

```
<?php
$books = [
    "Panduan Belajar PHP untuk Pemula",
    "Membangun Aplikasi Web dengan PHP",
    "Tutorial PHP dan MySQL",
    "Membuat Chat Bot dengan PHP"
];

echo "<h5>Judul Buku PHP:</h5>";
echo "<ul>";
foreach($books as $buku){
    echo "<li>$buku</li>";
}
echo "</ul>";

?>
```



- **Perulangan Bersarang**

- Perulangan bersarang adalah istilah untuk menyebut perulangan di dalam perulangan. Dalam bahasa inggris, perulangan bersarang disebut *nested loop*.
- Contoh perulangan bersarang:

```
<?php

for($i = 0; $i < 5; $i++){
    for($j = 0; $j < 10; $j++){
        echo "Ini perulangan ke ($i, $j)<br>";
    }
}

?>
```

```
<?php

$i = 0;
while($i < 10){
    for($j = 0; $j < 10; $j++){
        echo "Ini perulangan ke ($i, $j)<br>";
    }

    $i++;
}
```

localhost:8080/perulangan-bersarang.php

Ini perulangan ke (0, 0)
Ini perulangan ke (0, 1)
Ini perulangan ke (0, 2)
Ini perulangan ke (0, 3)
Ini perulangan ke (0, 4)
Ini perulangan ke (0, 5)
Ini perulangan ke (0, 6)
Ini perulangan ke (0, 7)
Ini perulangan ke (0, 8)
Ini perulangan ke (0, 9)
Ini perulangan ke (1, 0)
Ini perulangan ke (1, 1)
Ini perulangan ke (1, 2)
Ini perulangan ke (1, 3)
Ini perulangan ke (1, 4)
Ini perulangan ke (1, 5)
Ini perulangan ke (1, 6)

Array

- Array adalah salah satu struktur data yang berisi sekumpulan data dan memiliki indeks. Indeks digunakan untuk mengakses nilai array.
- Indeks array selalu dimulai dari nol (**0**).

"Hardisk 2TB"	"Flashdisk 32GB"	"Modem"
0	1	2

- Jadi, apabila kita ingin menampilkan “Hard disk 2TB”, maka kita harus mengambil indeks yang ke-0.

- **Membuat Array di PHP**
- Array di PHP dapat kita buat dengan fungsi **array()** dan tanda kurung kotak **[]**.

```
<?php

// membuat array kosong
$buah = array();
$hobi = [];

// membuat array sekaligus mengisinya
$minuman = array("Kopi", "Teh", "Jus Jeruk");
$makanan = ["Nasi Goreng", "Soto", "Bubur"];

// membuat array dengan mengisi indeks tertentu
$sanggota[1] = "Dian";
$sanggota[2] = "Muhar";
$sanggota[0] = "Petani Kode";
```

- array dapat kita isi dengan tipe data apa saja. Bahkan dicampur juga boleh.

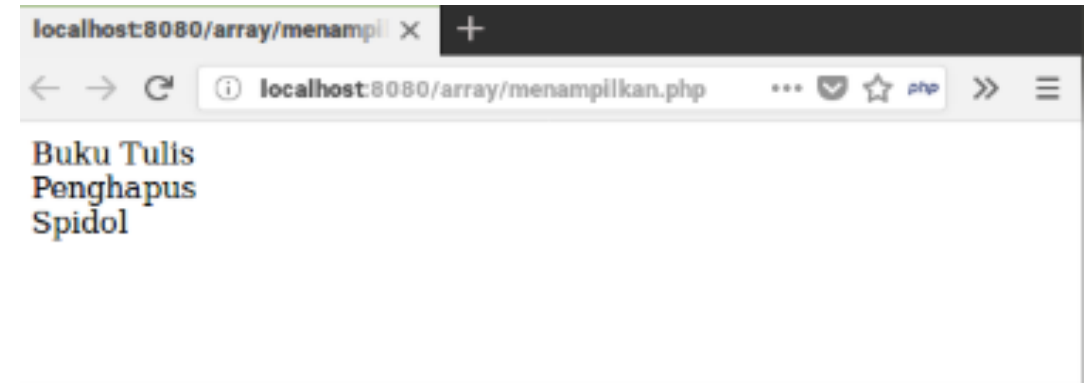
```
<?php

$item = ["Bunga", 123, 39.12, true];
```

- Menampilkan isi Array
- Untuk menampilkan isi array, kita bisa mengaksesnya melalui indeks.

```
<?php
// membuat array
$barang = ["Buku Tulis", "Penghapus", "Spidol"];

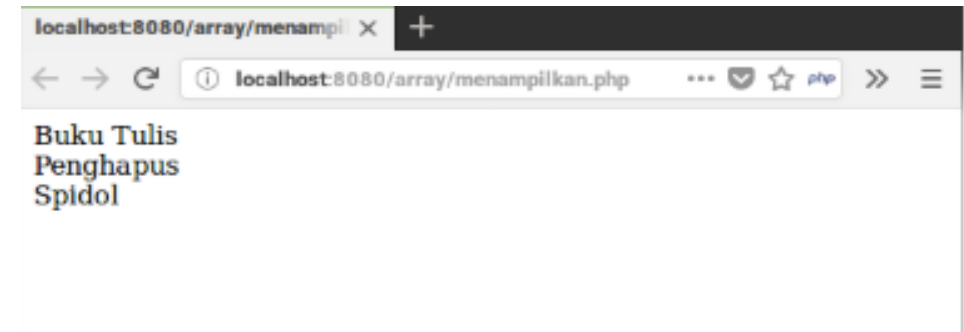
// menampilkan isi array
echo $barang[0]."<br>";
echo $barang[1]."<br>";
echo $barang[2]."<br>";
```



- Tapi cara ini kurang efektif, karena kita mencetak satu per satu. Nanti kalau datanya ada 1000, berarti harus ngetik perintah **echo** sebanyak 1000.
- Biasanya kita menggunakan perulangan.

```
<?php
// membuat array
$barang = ["Buku Tulis", "Penghapus", "Spidol"];

// menampilkan isi array dengan perulangan for
for($i=0; $i < count($barang); $i++){
    echo $barang[$i]."<br>";
}
```



- Kita bisa menggunakan fungsi **count()** untuk menghitung banyaknya isi array. Pada contoh di atas isi array sebanyak 3, maka perulangan akan dilakukan sebanyak 3x.

- Selain menggunakan perulangan **for**, kita juga bisa menggunakan perulangan **while** dan **foreach**.

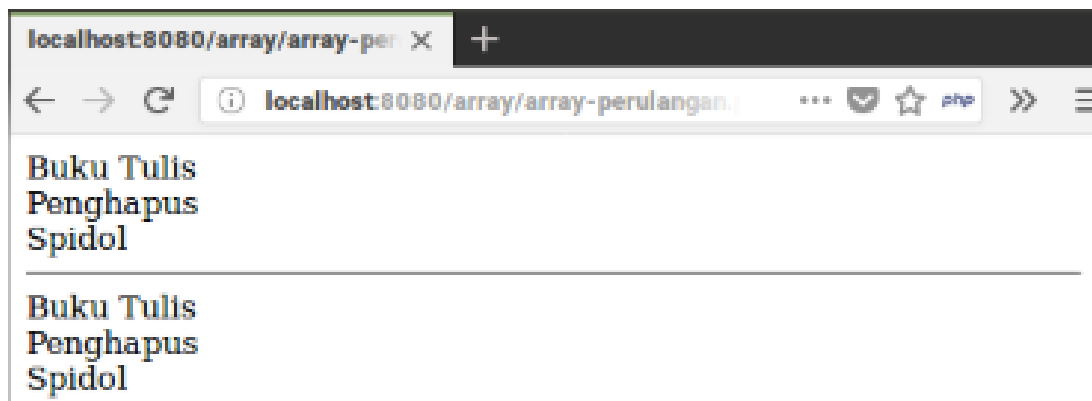
```
<?php

// membuat array
$barang = ["Buku Tulis", "Penghapus", "Spidol"];

// menampilkan isi array dengan perulangan foreach
foreach($barang as $isi){
    echo $isi."<br>";
}

echo "<hr>";

// menampilkan isi array dengan perulangan while
$i = 0;
while($i < count($barang)){
    echo $barang[$i]."<br>";
    $i++;
}
```



- **Menghapus isi Array**
- Untuk menghapus isi array, kita bisa menggunakan fungsi **unset()**. Fungsi ini juga dapat digunakan untuk menghapus variabel.

```
<?php

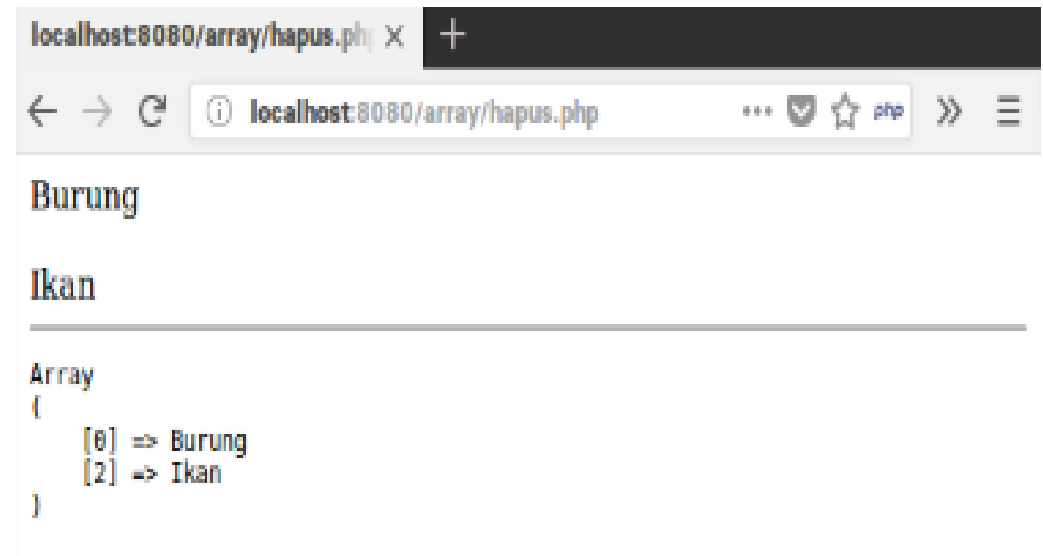
// membuat array
$hewan = [
    "Burung",
    "Kucing",
    "Ikan"
];

// menghapus kucing
unset($hewan[1]);

echo $hewan[0]."<br>";
echo $hewan[1]."<br>";
echo $hewan[2]."<br>";

echo "<hr>";

echo "<pre>";
print_r($hewan);
echo "</pre>";
```



- Pada contoh di atas, Kita menggunakan fungsi **print_r()** untuk menampilkan array secara mentah (*raw*). Biasanya fungsi ini digunakan untuk *debugging*.

- **Menambahkan isi Array**

- Ada dua cara yang bisa dilakukan untuk menambah isi array:

1. Mengisi langsung ke nomer indeks yang ingin ditambahkan

2. Mengisi langsung ke indeks terakhir

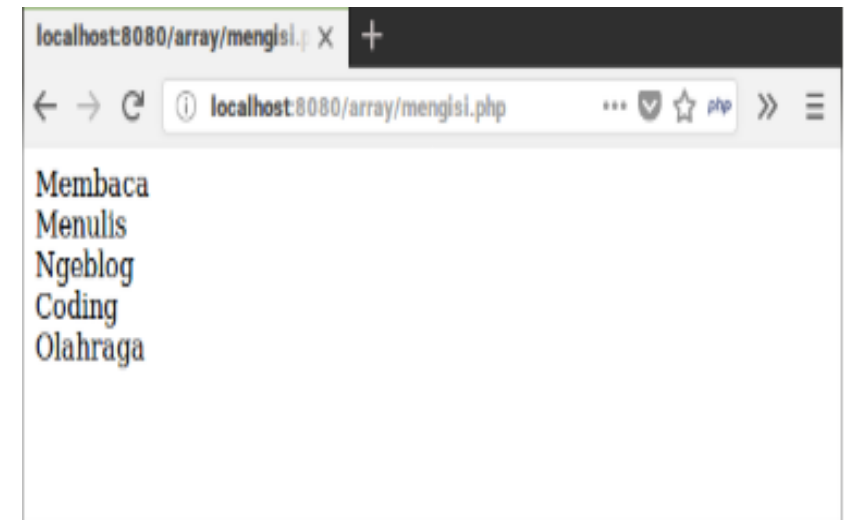
- Mari kita coba kedua-duanya.

```
<?php
// membuat array
$hobi = [
    "Membaca",
    "Menulis",
    "Ngeblog"
];

// menambahkan isi pada idenks ke-3
$hobi[3] = "Coding";

// menambahkan isi pada indeks terakhir
$hobi[] = "Olahraga";

// cetak array dengan perulangan
foreach($hobi as $hobiku){
    echo $hobiku."<br>";
}
?>
```



- Apabila kita menambahkan pada indeks yang sudah memiliki isi, maka isinya akan ditindih dengan yang baru.

```
<?php
// membuat array
$user = [
    "dian",
    "muhar",
    "petanikode"
];

// mengisi array pada indeks ke-1 ("muhar")
$user[1] = "ardianta";

// mencetak isi array
echo "<pre>";
print_r($user);
echo "</pre>";
?>
```

```
Array
(
    [0] => dian
    [1] => ardianta
    [2] => petanikode
)
```

- **Array Asosiatif**
- Array asosiatif adalah array yang indeksnya tidak menggunakan nomer atau angka. Indeks array asosiatif berbentuk kata kunci.

```
<?php
// membuat array asosiatif
$artikel = [
    "judul" => "Belajar Pemrograman PHP",
    "penulis" => "petanikode",
    "view" => 128
];

// mencetak isi array asosiatif
echo "<h2>".$artikel["judul"]."</h2>";
echo "<p>oleh: ".$artikel["penulis"]."</p>";
echo "<p>View: ".$artikel["view"]."</p>";
```



- Pada array asosiatif, kita menggunakan tanda => untuk mengasosiasikan sebuah kata kunci dengan isi array. Selain menggunakan tanda =>, kita juga bisa membuat array asosiatif seperti ini:

```
<?php
$email["subjek"] = "Apa Kabar?";
$email["pengirim"] = "dian@petanikode.com";
$email["isi"] = "Apa kabar? sudah lama tidak berjumpa";

echo "<pre>";
print_r($email);
echo "</pre>";
```

```
Array
(
    [subjek] => Apa Kabar?
    [pengirim] => dian@petanikode.com
    [isi] => Apa kabar? sudah lama tidak berjumpa
)
```

- **Array Multi Dimensi**

- Array multi dimensi adalah array yang memiliki dimensi lebih dari satu. Biasanya digunakan untuk membuat matriks, graph, dan struktur data rumit lainnya.

```
<?php
// ini adalah array dua dimensi
$matrik = [
    [2,3,4],
    [7,5,0],
    [4,3,8],
];

// cara mengakses isinya
echo $matrik[1][0]; //-> output: 7
```

- Mari kita coba contoh yang lain:

```
<?php
// membuat array 2 dimensi yang berisi array asosiatif
$artikel = [
    [
        "judul" => "Belajar PHP & MySQL untuk Pemula",
        "penulis" => "petanikode"
    ],
    [
        "judul" => "Tutorial PHP dari Nol hingga Mahir",
        "penulis" => "petanikode"
    ],
    [
        "judul" => "Membuat Aplikasi Web dengan PHP",
        "penulis" => "petanikode"
    ]
];

// menampilkan array
foreach($artikel as $post){
    echo "<h2>".$post["judul"]."</h2>";
    echo "<p>".$post["penulis"]."<p>";
    echo "<hr>";
}
```



Memahami Prosedur dan Fungsi

- Fungsi adalah sekumpulan instruksi yang dibungkus dalam sebuah blok. Fungsi dapat digunakan ulang tanpa harus menulis ulang instruksi di dalamnya.
- Fungsi pada PHP dapat dibuat dengan kata kunci **function**, lalu diikuti dengan nama fungsinya.

```
function namaFungsi(){  
    //...  
}
```

- Kode instruksi dapat di tulis di dalam kurung kurawal ({...}).

```
function perkenalan(){  
    echo "Assalamualaikmu, ";  
    echo "Perkenalkan, nama saya Ardianta<br/>";  
    echo "Senang berkenalan dengan anda<br/>";  
}
```

- Fungsi yang sudah dibuat tidak akan menghasilkan apapun kalau tidak dipanggil. Kita dapat memanggil fungsi dengan menuliskan namanya.

```
perknanalan();
```

- kode lengkapnya seperti ini:

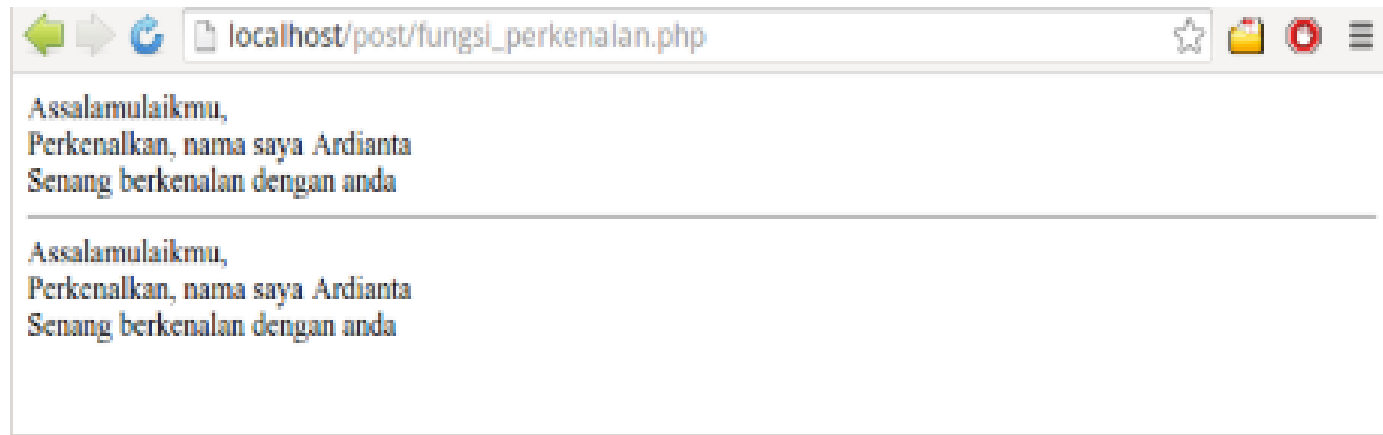
- Contoh :

```
<?php
// mmbuat fungsi
function perkenalan(){
    echo "Assalamualaikmu, ";
    echo "Perkenalkan, nama saya Ardianta<br/>";
    echo "Senang berkenalan dengan anda<br/>";
}

// memanggil fungsi yang sudah dibuat
perkenalan();

echo "<hr>";

// memanggilnya lagi
perkenalan();
?>
```



- **Fungsi dengan Parameter**
- Supaya instruksi yang di dalam fungsi lebih dinamis, kita dapat menggunakan parameter untuk memasukkan sebuah nilai ke dalam fungsi. Nilai tersebut akan diolah di dalam fungsi.
- Misalkan, pada contoh fungsi yang tadi, tidak mungkin nama yang dicetak adalah *ardianta* saja dan salam yang dipakai tidak selalu *assalamualaikum*. Maka, kita dapat menambahkan parameter menjadi seperti ini:

```
<?php
// mmbuat fungsi
function perkenalan($nama, $salam){
    echo $salam.", ";
    echo "Perkenalkan, nama saya ".$nama."<br/>";
    echo "Senang berkenalan dengan anda<br/>";
}

// memanggil fungsi yang sudah dibuat
perkenalan("Muhardian", "Hi");

echo "<hr>";

$saya = "Indry";
$ucapanSalam = "Selamat pagi";
// memanggilnya lagi
perkenalan($saya, $ucapanSalam);
?>
```



- **Paramater dengan Nilai Default**

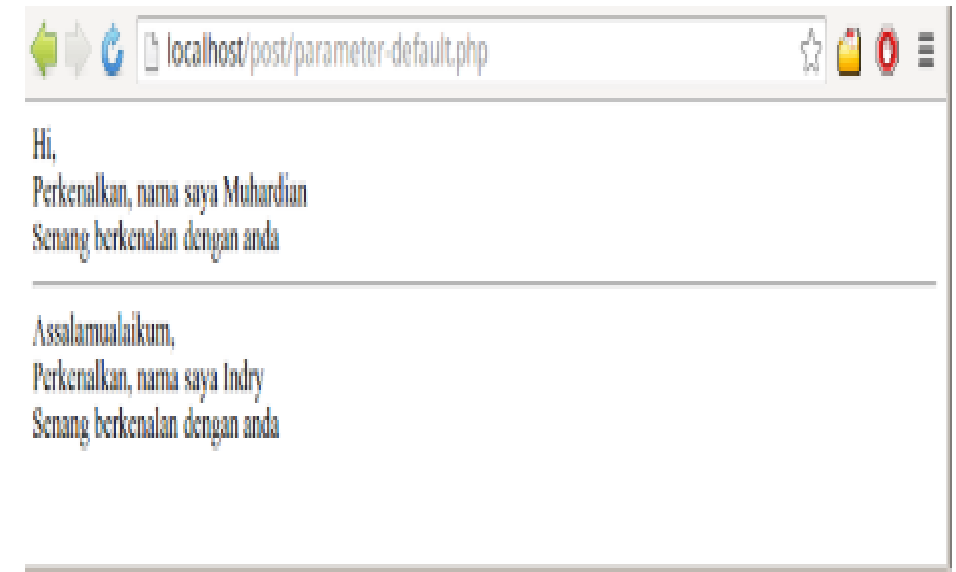
- Nilai *default* dapat kita berikan di parameter. Nilai *default* berfungsi untuk mengisi nilai sebuah parameter, kalau parameter tersebut tidak diisi nilainya.
- Misalnya: saya lupa mengisi parameter *salam*, maka program akan *error*. Oleh karena itu, kita perlu memberikan nilai *default* supaya tidak *error*.

```
<?php
// mmbuat fungsi
function perkenalan($nama, $salam="Assalamualaikum"){
    echo $salam.", ";
    echo "Perkenalkan, nama saya ".$nama."<br/>";
    echo "Senang berkenalan dengan anda<br/>";
}

// memanggil fungsi yang sudah dibuat
perkenalan("Muhardian", "Hi");

echo "<hr>";

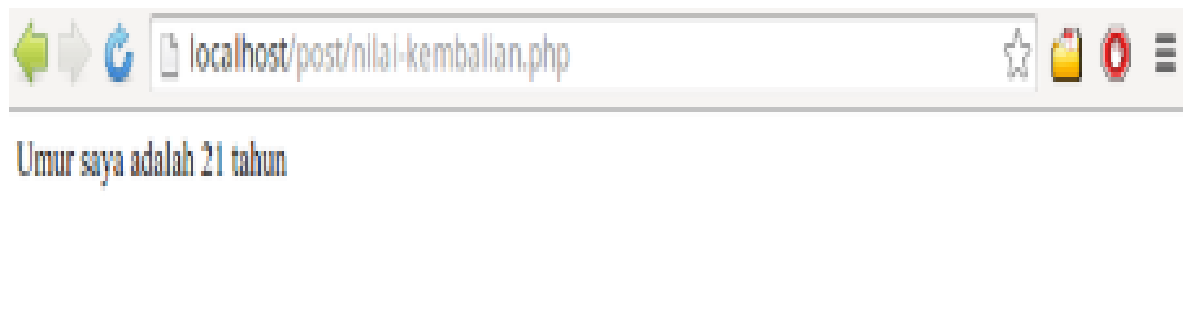
$saya = "Indry";
$ucapanSalam = "Selamat pagi";
// memanggilnya lagi tanpa mengisi parameter salam
perkenalan($saya);
?>
```



- **Fungsi yang Mengembalikan Nilai**
- Hasil pengolahan nilai dari fungsi mungkin saja kita butuhkan untuk pemrosesan berikutnya. Oleh karena itu, kita harus membuat fungsi yang dapat mengembalikan nilai.
- Pengembalian nilai dalam fungsi dapat menggunakan kata kunci **return**.

```
<?php
// membuat fungsi
function hitungUmur($thn_lahir, $thn_sekarang){
    $umur = $thn_sekarang - $thn_lahir;
    return $umur;
}

echo "Umur saya adalah ". hitungUmur(1994, 2015) ." tahun";
?>
```



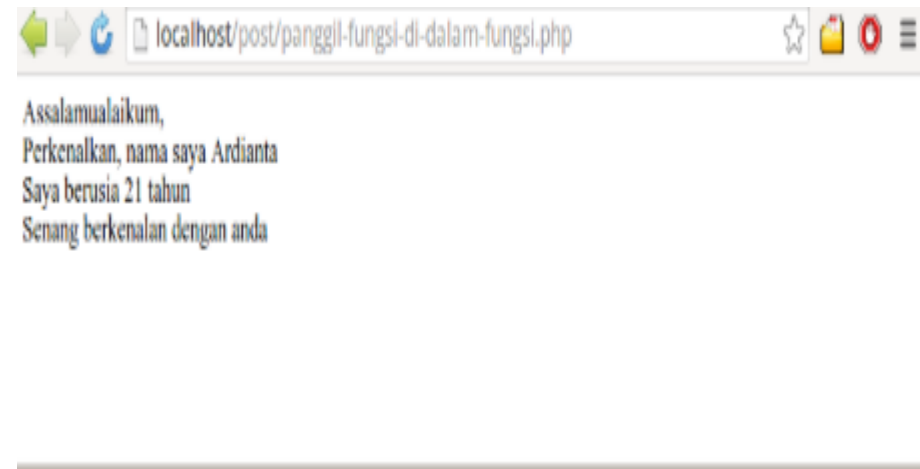
- **Memanggil Fungsi di dalam Fungsi**
- Fungsi yang sudah kita buat, dapat juga dipanggil di dalam fungsi lain.

```
<?php
// membuat fungsi
function hitungUmur($thn_Lahir, $thn_sekarang){
    $umur = $thn_sekarang - $thn_Lahir;
    return $umur;
}

function perkenalan($nama, $salam="Assalamualaikum"){
    echo $salam.", ";
    echo "Perkenalkan, nama saya ".$nama."<br/>";
    // memanggil fungsi lain
    echo "Saya berusia ". hitungUmur(1994, 2015) ." tahun<br/>";
    echo "Senang berkenalan dengan anda<br/>";
}

// memanggil fungsi perkenalan
perkenalan("Ardianta");

?>
```



- **Fungsi rekursif**
- Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri. Fungsi ini biasanya digunakan untuk menyelesaikan masalah seperti faktorial, bilangan fibonacci, pemrograman dinamis, dll.
- Contoh fungsi rekursif:

```
<?php

function faktorial($angka) {
    if ($angka < 2) {
        return 1;
    } else {
        // memanggil dirinya sendiri
        return ($angka * faktorial($angka-1));
    }
}

// memanggil fungsi
echo "faktorial 5 adalah " . faktorial(5);

?>
```



faktorial 5 adalah 120

Daftar Pustaka

- <https://www.petanikode.com/>